Introducción básica a Python - Econometría II
Universidad Nacional de Colombia

En este cuaderno aprenderemos los elementos fundamentales de Python mediante ejemplos sencillos. Puedes ejecutar cada celda en orden y cambiar los valores para experimentar.

Índice

Los siguientes enlaces permiten ir directamente a cada sección del cuaderno. Al terminar una sección, el enlace **Volver al índice** facilita el regreso a esta tabla.

Sección	Contenido
1	Introducción a Python
2	Tipos de datos básicos
3	Operaciones básicas
4	Tipos de datos compuestos
5	Estructuras de control
6	Funciones y programación imperativa
7	Clases y programación orientada a objetos
8	Alcance de las variables (*scope*)
9	Temas adicionales de Python
10	CPython y la biblioteca estándar
11	Librerías para análisis de datos y econometría
12	NumPy y álgebra lineal
—	Resumen

1. Introducción a Python

↑ Volver al índice

1.1 Introducción básica a Python

Python es un lenguaje de programación de propósito general, de alto nivel y de código abierto. Fue diseñado para favorecer la legibilidad: su sintaxis se aproxima al lenguaje humano, utiliza la indentación para delimitar bloques y permite expresar muchas tareas con relativamente pocas líneas de código. Un **lenguaje** establece las reglas para escribir programas; un **intérprete de Python** es el programa que lee esas instrucciones y las ejecuta.

Python se utiliza para automatizar tareas, procesar archivos, construir aplicaciones web y servicios, analizar datos, realizar cálculos científicos, estimar modelos estadísticos y econométricos, desarrollar inteligencia artificial y enseñar programación. Es común en universidades, entidades públicas, empresas, investigación y producción de software. Su amplia biblioteca estándar y su ecosistema de paquetes de terceros permiten extenderlo sin construir todas las herramientas desde cero.

Entre sus ventajas se encuentran la sintaxis clara, la disponibilidad en Windows, macOS y Linux, una comunidad extensa y la posibilidad de trabajar con scripts `.py`, notebooks `.ipynb` y documentos reproducibles `.qmd`. Python es además **multiparadigma**: permite combinar programación imperativa, orientada a objetos y funcional, entre otros enfoques.

1.2 Tipos de archivos de Python

Python puede aparecer dentro de varios formatos de archivo. Los tres que encontraremos con mayor frecuencia en el curso son `.py`, `.ipynb` y `.qmd`. Los tres pueden contener código Python, pero difieren en su estructura, forma de ejecución y propósito.

Archivos `.py`: scripts y módulos de Python

Un archivo `.py` es un archivo de **texto plano** que contiene código fuente de Python. Sus instrucciones se ejecutan normalmente de arriba hacia abajo con un intérprete, por ejemplo mediante `python analisis.py`. También puede importarse desde otro programa como un **módulo**, lo que permite reutilizar funciones y clases.

Los archivos `.py` son apropiados para automatizaciones, programas completos, módulos reutilizables, pruebas y código que se ejecutará repetidamente. No almacenan dentro del archivo los resultados producidos al ejecutarlos. Al ser texto plano, son fáciles de revisar, comparar y administrar con sistemas de control de versiones como Git.

```
# analisis.py: define una función reutilizable y después la ejecuta.
def calcular_promedio(valores):
    """Calcula el promedio de una colección de valores."""
    return sum(valores) / len(valores)


# Muestra un resultado cuando se ejecuta el script.
print(calcular_promedio([3.5, 4.0, 4.5]))
```

Archivos `.ipynb`: notebooks de Jupyter

Un archivo `.ipynb` es un documento de **Jupyter Notebook**. Internamente utiliza el formato JSON y organiza el contenido en celdas de código, Markdown y otros tipos. Puede guardar código, explicaciones, ecuaciones, gráficos, tablas, resultados de ejecución y metadatos del kernel en un mismo archivo. Este cuaderno es un ejemplo de `.ipynb`.

El código se ejecuta mediante un **kernel** y las variables permanecen en memoria entre celdas. Esto favorece la exploración, la docencia y el análisis interactivo, pero exige atender al orden de ejecución: el orden visible de las celdas puede no coincidir con el orden en que se ejecutaron. Reiniciar el kernel y ejecutar todo desde el principio ayuda a comprobar que el análisis sea reproducible.

Los notebooks son excelentes para experimentar y combinar narrativa con resultados inmediatos. Sin embargo, su estructura JSON y las salidas incrustadas pueden producir archivos más grandes y diferencias de Git menos fáciles de revisar que las de un `.py`.

Archivos `.qmd`: documentos de Quarto

Un archivo `.qmd` es un documento de **Quarto Markdown** en texto plano. Combina una cabecera YAML, texto Markdown, ecuaciones, citas y bloques de código ejecutable. No es un formato exclusivo de Python: también puede contener R, Julia, Observable JavaScript y otros lenguajes. Cuando incluye bloques `{python}`, Quarto puede usar Jupyter para ejecutar el código durante la renderización.

Su objetivo principal es producir documentos reproducibles y presentables en formatos como HTML, PDF, Word o presentaciones. Para procesarlo se necesita instalar Quarto y disponer del intérprete, kernel y paquetes requeridos por el código. Como su fuente es texto plano, suele integrarse mejor con Git que un notebook con salidas almacenadas.

```
---
title: "Informe econométrico"
format: html
jupyter: python3
```

````
---

El promedio se calcula con el siguiente bloque:

```{python}
# Este código se ejecuta cuando Quarto procesa el documento.
valores = [3.5, 4.0, 4.5]
sum(valores) / len(valores)
```
````

Comparación de los formatos

Característica	`.py`	`.ipynb`	`.qmd`
Estructura	Texto plano con código Python	JSON organizado en celdas	Texto plano con Markdown, YAML y bloques de código
Ejecución	Intérprete de Python	Kernel de Jupyter, celda por celda	Motor de Quarto, frecuentemente mediante Jupyter para Python
Narrativa y ecuaciones	Posibles principalmente como comentarios o cadenas	Celdas Markdown integradas	Parte central del documento
Resultados almacenados en el archivo fuente	No	Sí, opcionalmente	Normalmente se generan al renderizar
Lenguajes	Python	El lenguaje del kernel	Python, R, Julia y otros
Uso recomendado	Scripts, módulos, automatización y aplicaciones	Exploración, enseñanza y análisis interactivo	Informes, artículos, libros y presentaciones reproducibles
Control de versiones	Muy sencillo	Posible, pero el JSON y las salidas generan diferencias más complejas	Sencillo por ser texto plano

En resumen: utilice `.py` cuando el código reutilizable sea el producto principal; `.ipynb` cuando necesite experimentar interactivamente y observar resultados por celdas; y `.qmd` cuando el producto principal sea un informe o publicación reproducible que integre código, resultados y narrativa.

1.3 Programas para trabajar localmente con Python

Para ejecutar Python en un computador se necesitan, como mínimo, un **intérprete** y alguna herramienta donde escribir instrucciones. En este curso emplearemos dos programas principales:

1. **Visual Studio Code (VS Code):** es el editor donde escribiremos, organizaremos y revisaremos archivos `.py` y notebooks `.ipynb`. El editor no incluye por sí solo el intérprete. La extensión **Python** de Microsoft agrega selección de intérpretes, autocompletado, ejecución, depuración y pruebas; la extensión **Jupyter** permite trabajar con notebooks.
2. **Anaconda Distribution:** proporciona una distribución de Python que incluye un intérprete, Jupyter y numerosas herramientas de análisis de datos. También incorpora **conda**, un administrador de paquetes, dependencias y ambientes.

La paleta de comandos: `Ctrl + Shift + P`

En Windows, `Ctrl + Shift + P` abre la **paleta de comandos** de VS Code. Esta paleta funciona como un buscador de acciones: permite encontrar y ejecutar casi todos los comandos del editor y de sus extensiones sin recorrer los menús. Después de abrirla, se escribe una parte del nombre del comando, se selecciona la opción con las flechas y se presiona `Enter`; `Esc` cierra la paleta. La tecla `F1` ofrece el mismo acceso.

> **No se deben confundir:** `Ctrl + P` busca y abre archivos del proyecto, mientras que `Ctrl +`

> `Shift + P` busca y ejecuta comandos.

Dos comandos importantes

1. **`Developer: Reload Window`:** recarga la ventana actual de VS Code y vuelve a iniciar su interfaz y sus extensiones, de manera semejante a reiniciar esa ventana sin tener que volver a abrir manualmente la carpeta del proyecto. Es útil después de instalar o actualizar una extensión, aplicar ciertos cambios de configuración o cuando la interfaz no refleja el estado reciente de los archivos. No ejecuta el código ni selecciona un ambiente de Python. Antes de usarlo, conviene guardar únicamente los cambios que se quieran conservar; si el notebook abierto es una versión desactualizada que podría sobrescribir la versión guardada en disco, se debe cerrar **sin guardar** y volver a abrir el archivo.
2. **`Python: Select Interpreter`:** permite elegir el ejecutable de Python o el ambiente virtual que VS Code utilizará para ejecutar y depurar archivos `.py`, además de proporcionar autocompletado, análisis del código y otras funciones de la extensión Python. Para usarlo: abrir la paleta con `Ctrl + Shift + P`, escribir `Python: Select Interpreter` y escoger, por ejemplo, el ambiente de conda `econometria2`. El intérprete seleccionado suele aparecer en la barra de estado.

> **Intérprete y kernel no son exactamente lo mismo:** en un archivo `.py` se selecciona el intérprete de Python; en un notebook `.ipynb` también se debe revisar el **kernel** en la esquina superior derecha. Ambos deberían apuntar al ambiente donde están instalados los paquetes que necesita el curso.

Atajos principales de VS Code en Windows

Atajo	Acción
`Ctrl + Shift + P` o `F1`	Abrir la paleta de comandos.
`Ctrl + P`	Buscar y abrir rápidamente un archivo del proyecto.
`Ctrl + S`	Guardar el archivo activo.
`Ctrl + F` / `Ctrl + H`	Buscar / buscar y reemplazar dentro del archivo activo.
`Ctrl + Shift + F`	Buscar texto en todos los archivos del proyecto.
`Ctrl + /`	Comentar o descomentar la línea o selección.
`Alt + ↑` / `Alt + ↓`	Mover la línea o selección hacia arriba o abajo.
`Shift + Alt + ↑` / `Shift + Alt + ↓`	Duplicar la línea o selección.
`Shift + Alt + F`	Formatear el documento.
`Ctrl + ` `	Mostrar u ocultar la terminal integrada.
`Ctrl + B`	Mostrar u ocultar la barra lateral.
`Ctrl + J`	Mostrar u ocultar el panel inferior.
`F5`	Iniciar la ejecución con depuración.
`Ctrl + F5`	Ejecutar sin depuración.
`Ctrl + K`, luego `Ctrl + S`	Abrir la lista de atajos y personalizarlos.

En los notebooks de Jupyter también resultan especialmente útiles estos atajos:

Atajo	Acción en un notebook
`Ctrl + Enter`	Ejecutar la celda seleccionada y conservar el foco en ella.
`Shift + Enter`	Ejecutar la celda y avanzar a la siguiente.
`Alt + Enter`	Ejecutar la celda e insertar una nueva debajo.
`Esc` / `Enter`	Pasar al modo comando / entrar al modo edición de la celda.
`A` / `B` en modo comando	Insertar una celda arriba / abajo.
`M` / `Y` en modo comando	Cambiar la celda a Markdown / código.
`D`, `D` en modo comando	Eliminar la celda seleccionada.

Estos son los atajos predeterminados para Windows. Pueden cambiar según el sistema operativo, la distribución del teclado o las personalizaciones del usuario; la referencia completa se encuentra en la documentación oficial de atajos de VS Code.

Ambientes virtuales con Anaconda y conda

Un **paquete de terceros** es código que no pertenece a la biblioteca estándar de Python, como NumPy, pandas o statsmodels. `conda`, incluido con Anaconda, permite instalar estos paquetes y administrar sus dependencias.

Un **ambiente virtual** es un espacio aislado que contiene su propio intérprete de Python y su propia colección de paquetes y versiones. El ambiente no es una máquina virtual ni otro sistema operativo: es una carpeta administrada por conda cuyos programas y bibliotecas se utilizan al activarla. Anaconda crea inicialmente un ambiente general llamado `base`, pero es recomendable crear un ambiente separado para cada curso o proyecto.

Los ambientes virtuales sirven principalmente para:

- **Evitar conflictos:** un proyecto puede necesitar una versión de pandas y otro una versión diferente, sin que una instalación modifique la otra.
- **Aislar experimentos:** instalar, actualizar o retirar un paquete dentro de un ambiente no altera los demás ambientes.
- **Facilitar la reproducibilidad:** se pueden registrar y compartir las versiones utilizadas para que otras personas reconstruyan un entorno semejante.
- **Mantener organizado el computador:** cada proyecto utiliza únicamente el intérprete y las dependencias que necesita.
- **Proteger el ambiente `base`:** reservarlo para administrar conda reduce la posibilidad de dañar herramientas necesarias para otros trabajos.

Creación y uso paso a paso

Los siguientes comandos se pueden ejecutar en **Anaconda Prompt** o en una terminal de VS Code donde conda esté disponible:

```
# Comprueba que conda esté instalado y disponible en la terminal.
conda --version

# Muestra los ambientes existentes; un asterisco identifica el ambiente activo.
conda env list

# Crea el ambiente econometria2 con una versión específica de Python.
conda create --name econometria2 python=3.12

# Activa el ambiente: desde este momento la terminal usa su Python y sus paquetes.
conda activate econometria2

# Comprueba la versión del intérprete activo.
python --version

# Instala dentro del ambiente las librerías utilizadas en el curso.
```

```
conda install numpy pandas matplotlib seaborn scipy statsmodels jupyter ipykernel

# Muestra todos los paquetes instalados en el ambiente activo.
conda list

# Sale del ambiente cuando se termina de trabajar.
conda deactivate
```

Al ejecutar `conda create`, conda muestra los paquetes que instalará y normalmente solicita confirmación con `Proceed ([y]/n)?`; se escribe `y` y se presiona `Enter`. La opción `--name` —abreviada como `-n`— establece el nombre del ambiente. Especificar `python=3.12` fija esa versión; escribir solamente `python` permite que conda elija una versión compatible disponible.

La activación con `conda activate econometria2` modifica temporalmente las rutas de la terminal para que los comandos `python`, `pip` y otras herramientas apunten al ambiente seleccionado. Usualmente el nombre aparece entre paréntesis al comienzo del prompt, por ejemplo `(econometria2)`. La activación se aplica a esa terminal: una terminal nueva puede requerir que se active otra vez.

Si el ambiente deja de ser necesario, primero se debe salir de él y luego eliminarlo explícitamente:

```
# Desactiva el ambiente antes de eliminarlo.
conda deactivate

# Elimina el ambiente y todos los paquetes contenidos en él.
conda remove --name econometria2 --all

# Confirma que el ambiente ya no aparezca en la lista.
conda env list
```

> **Advertencia:** el comando de eliminación solo debe usarse cuando realmente se quiera borrar el ambiente. No es necesario eliminarlo al terminar una sesión; normalmente basta con ejecutar `conda deactivate`.

Después de crear el ambiente, el flujo habitual será: abrir la carpeta del curso en VS Code, ejecutar **`Python: Select Interpreter`** y elegir `econometria2`; después, abrir el notebook y seleccionar el kernel del mismo ambiente. Es importante verificar ambos, porque un paquete instalado en un ambiente no aparece automáticamente en los demás. La documentación oficial de conda explica también cómo actualizar, exportar, compartir y reconstruir ambientes.

1.4 Comentarios en Python

Los **comentarios** documentan la intención del código, explican decisiones relevantes y facilitan su mantenimiento. El intérprete no ejecuta el texto que aparece después de `#`. Un comentario puede ocupar una línea completa o aparecer al final de una instrucción. Debe explicar el **porqué** o resumir un bloque, mantenerse actualizado y evitar repetir de manera obvia lo que ya expresa el código.

```
# Este comentario ocupa una línea completa.
tasa_interes = 0.05  # Este es un comentario al final de una instrucción.
```

Python no posee un delimitador especial para comentarios de varias líneas. La forma recomendada es escribir varias líneas consecutivas, cada una iniciada con `#`:

```
# Este comentario explica un procedimiento que requiere
# varias líneas para ser comprendido con claridad.
resultado = 100 * (1 + tasa_interes)
```

A veces se usan comillas triples (`"""..."""` o `'''...'''`) como si fueran comentarios multilínea, pero técnicamente crean una **cadena de texto**. Cuando esa cadena es la primera instrucción de un módulo, función, clase o método, se convierte en un **docstring** y Python la conserva como documentación accesible mediante `help()` o `.__doc__`. Para comentar temporalmente un bloque se prefieren varias líneas con `#`; para documentar una interfaz se utiliza un docstring.

```
def calcular_total(precio, impuesto):
    """Calcula el precio total después de aplicar un impuesto."""
    # Devuelve el resultado al código que llamó la función.
    return precio * (1 + impuesto)
```

1.5 PEP 8 y escritura legible de código

El nombre correcto es **PEP 8 — Style Guide for Python Code**. PEP significa *Python Enhancement Proposal*. PEP 8 reúne convenciones para que el código sea legible y consistente; no cambia lo que calcula un programa, pero sí facilita que otras personas —y nosotros mismos en el futuro— puedan comprenderlo, revisarlo y mantenerlo.

Entre sus recomendaciones más útiles se encuentran:

- Usar **cuatro espacios por nivel de indentación** y no mezclar espacios con tabulaciones.
- Nombrar variables y funciones con `snake_case`, clases con `CamelCase` y constantes con `MAYUSCULAS_CON_GUIONES_BAJOS`.
- Rodear con espacios los operadores (`total = precio + impuesto`) y dejar un espacio después de cada coma.
- Escribir normalmente una importación por línea y agrupar primero la biblioteca estándar, luego paquetes de terceros y finalmente módulos propios.
- Separar funciones y clases con líneas en blanco y dividir expresiones extensas dentro de paréntesis.
- Mantener comentarios correctos y comprensibles. La guía tradicional recomienda hasta 79 caracteres para líneas de código y 72 para comentarios o docstrings, aunque un proyecto puede adoptar otro límite de forma consistente.

Herramientas como **Black** pueden formatear el código automáticamente y herramientas como **Ruff** pueden detectar problemas de estilo y errores frecuentes. Sin embargo, un formateador no reemplaza buenos nombres, una estructura clara ni comentarios pertinentes. PEP 8 también recuerda que la consistencia dentro de un proyecto y la legibilidad prevalecen cuando una regla concreta no resulta adecuada.

1.6 Paradigmas de programación

Un **paradigma de programación** es una manera general de organizar el razonamiento y expresar la solución de un problema. No siempre corresponde a un lenguaje exclusivo: muchos lenguajes modernos son multiparadigma y permiten combinar varios estilos.

1. **Programación imperativa:** indica **cómo** resolver una tarea mediante una secuencia de instrucciones que consulta o modifica el estado del programa. Asignaciones, condicionales, ciclos y llamadas de funciones aparecen en un orden explícito. C, Python, R y Java permiten este estilo. En la sección de funciones construiremos una estimación MCO como una secuencia de pasos.
2. **Programación orientada a objetos (POO):** organiza el programa alrededor de objetos que reúnen estado —atributos— y comportamiento —métodos—. Favorece encapsulación, reutilización y especialización mediante clases y herencia. Java, C++, C#, Ruby y Python ofrecen POO. La clase `MCO` del cuaderno será un ejemplo.
3. **Programación funcional:** trata las funciones como valores, favorece funciones puras —un mismo argumento produce el mismo resultado sin modificar estado externo—, composición e inmutabilidad. Haskell, Lisp, Clojure y F# tienen una orientación funcional fuerte; Python, R, JavaScript y Scala también permiten aplicar muchas de estas ideas.

4. **Programación declarativa:** expresa **qué resultado** se desea sin detallar necesariamente cada paso de control. SQL describe qué datos consultar y el motor decide cómo obtenerlos; Prolog expresa hechos y reglas para que el sistema busque soluciones. La programación funcional también suele considerarse una familia declarativa, aunque aquí la distinguimos para resaltar sus técnicas particulares.

Elegir un paradigma depende del problema. Un análisis puede cargar y limpiar datos imperativamente, representar modelos mediante objetos, transformar columnas con funciones y consultar una base de datos de forma declarativa.

1.7 Lenguajes compilados e interpretados

En un **lenguaje compilado**, un compilador traduce normalmente el programa completo a código máquina o a una representación intermedia antes de su ejecución. El proceso puede detectar ciertos errores anticipadamente y producir un ejecutable optimizado. C, C++, Rust y Go son ejemplos que suelen compilarse directamente a código nativo.

En un **lenguaje interpretado**, un intérprete participa durante la ejecución y permite ejecutar instrucciones de manera interactiva, algo especialmente útil en consolas y notebooks. **Python y R se consideran lenguajes interpretados**, al igual que JavaScript en sus usos habituales. Esto permite experimentar rápidamente, aunque algunos errores solamente aparecen cuando la ejecución alcanza la instrucción problemática.

La distinción no es absoluta. La implementación habitual, CPython, compila el código fuente `.py` a un formato intermedio llamado ***bytecode*** y luego la máquina virtual de Python lo ejecuta. Java suele compilar a *bytecode* para la JVM, y varios intérpretes modernos incorporan compilación JIT (*just in time*). Por tanto, «compilado» e «interpretado» describen estrategias de implementación y ejecución, no dos categorías completamente excluyentes ni una garantía automática de velocidad.

1.8 Tipado estático y dinámico

En los lenguajes de **tipado estático**, el tipo de una variable o expresión se determina y verifica principalmente antes de ejecutar el programa. Esto permite detectar anticipadamente varias incompatibilidades. C, C++, Rust, Java y C# usan tipado estático; TypeScript agrega verificación estática al ecosistema de JavaScript.

En los lenguajes de **tipado dinámico**, los objetos conservan sus tipos y las comprobaciones principales ocurren durante la ejecución. Un mismo nombre puede referirse en momentos diferentes a objetos de tipos distintos. **Python es un lenguaje de tipado dinámico**, al igual que R, JavaScript, Ruby y Julia. Dinámico no significa que los valores carezcan de tipo: `10` es un `int` y `"diez"` es un `str`; significa que la variable `dato` no queda restringida permanentemente a uno de ellos.

```
# El nombre dato referencia primero un entero.
dato = 10
print(type(dato))

# El mismo nombre puede referenciar después una cadena.
dato = "diez"
print(type(dato))

# Una anotación comunica la intención, pero Python no la impone por sí solo.
edad: int = 20
```

Python admite **anotaciones de tipo** opcionales, como `edad: int`, que mejoran documentación, autocompletado y análisis con herramientas como mypy o Pyright. El intérprete estándar no impide automáticamente asignar después otro tipo. Además, Python suele describirse como de tipado **fuerte** porque evita muchas conversiones implícitas ambiguas; por ejemplo, sumar directamente `1 + "2"` produce `TypeError`. Tipado dinámico y tipado débil no son sinónimos.

---

2. Tipos de datos básicos

↑ Volver al índice

Una **variable** es un nombre que utilizamos para referirnos a un valor u objeto. Python reconoce automáticamente el tipo de dato asociado con ese nombre.

Variables y operador de asignación =

El símbolo = es el **operador de asignación**. Una instrucción con la forma `nombre = expresión` hace que Python evalúe primero la expresión situada a la derecha y después asocie su resultado con el nombre ubicado a la izquierda. Por eso, en programación normalmente se lee de derecha a izquierda: `edad = 20` se puede leer como **«asigne el valor 20 a la variable edad»** o **«guarde 20 con el nombre edad»**. No se lee simplemente «edad es igual a 20» en el sentido de una igualdad matemática.

```
# Asigna el número entero 20 al nombre edad.
edad = 20

# Evalúa primero 20 + 1 y asigna el resultado, 21, al nombre edad_siguiente.
edad_siguiente = edad + 1

# Evalúa el valor actual de contador, le suma uno y reasigna el resultado.
contador = 0
contador = contador + 1
```

Una variable puede recibir una nueva asignación: el valor anterior deja de ser el valor asociado con ese nombre. Esto explica por qué `contador = contador + 1` es válido en programación, aunque no tendría sentido como ecuación algebraica: el lado derecho se calcula con el valor actual y luego el resultado se vuelve a asignar al nombre de la izquierda.

> **Importante:** = **asigna** un valor, mientras que == **compara** dos valores y devuelve `True` o `False`. Por ejemplo, `edad = 20` realiza una asignación, pero `edad == 20` pregunta si el valor asociado con `edad` es igual a `20`.

Antes de estudiar los tipos básicos, utilizaremos dos funciones incorporadas de Python que no requieren importar ninguna librería.

Las funciones `print()` y `type()`

Una función se utiliza escribiendo su nombre seguido de paréntesis. Dentro de los paréntesis se colocan los argumentos, es decir, los valores con los que trabajará la función.

- `print()` muestra información en la salida de la celda. Puede recibir texto, variables o varios valores separados por comas. Por defecto, separa los valores con un espacio y termina con un salto de línea. Los argumentos opcionales `sep` y `end` permiten cambiar esos comportamientos.
- `type()` recibe un objeto y devuelve su tipo. Por ejemplo, `type(20)` devuelve `int`. Para mostrar claramente ese resultado podemos anidar las funciones y escribir `print(type(20))`: primero se evalúa `type(20)` y luego `print()` muestra el resultado.

```
dato = 20

print(dato)                      # Muestra solamente el valor
print("Valor:", dato)            # Combina texto y una variable
print("Tipo:", type(dato))       # Muestra el tipo de la variable
print("A", "B", sep=" - ")     # Cambia el separador
```

2.1 Enteros (`int`)

Los enteros son números sin parte decimal.

```
# Guarda un número entero en la variable edad.
edad = 20

# Muestra el valor almacenado y el tipo reconocido por Python.
print("Valor de edad:", edad)
print("Tipo de edad:", type(edad))
```

```
Valor de edad: 20
Tipo de edad: <class 'int'>
```

2.2 Números decimales (`float`)

Los valores de tipo `float` tienen una parte decimal. En Python se utiliza un punto como separador decimal.

```
# Guarda un número decimal de tipo float.
estatura = 1.72

# Comprueba el valor y su tipo mediante print() y type().
print("Valor de estatura:", estatura)
print("Tipo de estatura:", type(estatura))
```

```
Valor de estatura: 1.72
Tipo de estatura: <class 'float'>
```

2.3 Cadenas de texto (`str`)

Una cadena de texto se escribe entre comillas simples o dobles.

```
# Crea una cadena de texto delimitada por comillas.
nombre = "Ana"

# Muestra el contenido y confirma que la variable es de tipo str.
print("Valor de nombre:", nombre)
print("Tipo de nombre:", type(nombre))
```

```
Valor de nombre: Ana
Tipo de nombre: <class 'str'>
```

2.4 Booleanos (`bool`)

Un booleano solo puede tomar los valores `True` o `False`, escritos con mayúscula inicial. Se utiliza para representar condiciones, responder preguntas lógicas y controlar qué instrucciones debe ejecutar un programa. Las comparaciones también producen resultados booleanos.

```
# Los booleanos representan valores lógicos verdaderos o falsos.
es_estudiante = True
tiene_beca = False
# Una comparación también produce un valor booleano.
mayor_de_edad = 20 >= 18

# Muestra cada valor booleano junto con el tipo que devuelve type().
print("Valor de es_estudiante:", es_estudiante)
print("Tipo de es_estudiante:", type(es_estudiante))
print("Valor de tiene_beca:", tiene_beca)
print("Tipo de tiene_beca:", type(tiene_beca))
print("Resultado de la comparación 20 >= 18:", mayor_de_edad)
print("Tipo del resultado:", type(mayor_de_edad))
```

```
Valor de es_estudiante: True
Tipo de es_estudiante: <class 'bool'>
Valor de tiene_beca: False
Tipo de tiene_beca: <class 'bool'>
Resultado de la comparación 20 >= 18: True
Tipo del resultado: <class 'bool'>
```

2.5 Valor nulo (`None`)

El valor especial `None` representa la ausencia de un valor o indica que un dato todavía no está disponible. No es lo mismo que `0`, `False` ni una cadena vacía. Su tipo es `NoneType`. Para comprobar si una variable contiene `None`, se recomienda utilizar `is None`.

```
# None representa explícitamente la ausencia de un valor.
dato_faltante = None

# El operador is verifica si la variable referencia exactamente a None.
print("Valor de dato_faltante:", dato_faltante)
print("Tipo de dato_faltante:", type(dato_faltante))
print("¿El dato es None?", dato_faltante is None)
```

```
Valor de dato_faltante: None
Tipo de dato_faltante: <class 'NoneType'>
¿El dato es None? True
```

2.6 Repaso de los tipos de datos básicos

Podemos comparar en una misma celda los tipos de todas las variables anteriores mediante `print()` y `type()`.

```
# Repasa los tipos de las variables creadas en las celdas anteriores.
print("edad:", type(edad))
print("estatura:", type(estatura))
print("nombre:", type(nombre))
print("es_estudiante:", type(es_estudiante))
print("dato_faltante:", type(dato_faltante))
```

```
edad: <class 'int'>
estatura: <class 'float'>
nombre: <class 'str'>
es_estudiante: <class 'bool'>
dato_faltante: <class 'NoneType'>
```

---

3. Operaciones básicas

↑ Volver al índice

3.1 Operaciones aritméticas

Python permite sumar, restar, multiplicar, dividir y realizar otras operaciones numéricas.

```
# Define dos operandos enteros para comparar operaciones aritméticas.
a = 10
b = 3

# Aplica los operadores aritméticos básicos y muestra sus resultados.
print("Suma:", a + b)
print("Resta:", a - b)
print("Multiplicación:", a * b)
print("División:", a / b)
print("División entera:", a // b)
```

```
print("Residuo:", a % b)
print("Potencia:", a ** b)
```

```
Suma: 13
Resta: 7
Multiplicación: 30
División: 3.3333333333333335
División entera: 3
Residuo: 1
Potencia: 1000
```

3.2 Comparaciones y operaciones lógicas

Las comparaciones evalúan una relación y producen un valor booleano: `True` o `False`. Los operadores lógicos `and`, `or` y `not` permiten combinar o negar esas condiciones.

3.2.1 Igualdad == y desigualdad !=

El operador == pregunta si dos valores son **iguales**. Devuelve `True` cuando lo son y `False` cuando no lo son. El operador `!=` pregunta si son **diferentes** y produce el resultado opuesto. Estos operadores no modifican los valores comparados.

Es fundamental distinguir == de =: un solo signo igual es el operador de **asignación** y sirve para asociar un nombre con un valor, como en `nota = 4.0`; dos signos iguales forman una **comparación**, como en `nota == 4.0`. Por su parte, `is` estudia si dos nombres se refieren al mismo objeto y no debe utilizarse como sustituto general de ==. Una excepción habitual es `dato is None`.

Expresión	Pregunta	Ejemplo	Resultado
`a == b`	¿a y b tienen valores iguales?	`5 == 5`	`True`
`a != b`	¿a y b tienen valores diferentes?	`5 != 5`	`False`
`a == b`	La igualdad también considera los tipos y sus reglas de comparación	`5 == "5"`	`False`

3.2.2 Tabla de verdad de and

La conjunción `x and y` es `True` **solamente cuando ambas condiciones son verdaderas**. Si una de ellas es falsa, la expresión completa es falsa. Puede interpretarse como «se deben cumplir todos los requisitos».

x	y	x and y
`False`	`False`	`False`
`False`	`True`	`False`
`True`	`False`	`False`
`True`	`True`	`True`

3.2.3 Tabla de verdad de or

La disyunción `x or y` es `True` cuando **al menos una condición es verdadera**. Únicamente es falsa cuando las dos condiciones son falsas. Puede interpretarse como «basta con cumplir uno de los requisitos». En programación, `or` es inclusivo: también es verdadero cuando ambas condiciones son verdaderas.

x	y	x or y
False	False	False
False	True	True
True	False	True
True	True	True

El operador `not` niega una condición: `not True` es `False` y `not False` es `True`. Python evalúa primero las comparaciones, luego `not`, después `and` y finalmente `or`; aun así, se recomienda usar paréntesis cuando una expresión compleja pueda resultar ambigua. Además, `and` y `or` usan **evaluación de cortocircuito**: `and` deja de evaluar al encontrar una condición falsa y `or` al encontrar una verdadera.

```
# Compara valores sin confundir igualdad (==) con asignación (=).
numero = 5
texto = "5"

print("¿5 es igual a 5?:", numero == 5)
print("¿5 es diferente de 4?:", numero != 4)
print("¿El número 5 es igual al texto '5'?:", numero == texto)

# Verifica directamente las cuatro combinaciones de la tabla de and.
print("False and False:", False and False)
print("False and True :", False and True)
print("True  and False:", True and False)
print("True  and True :", True and True)

# Verifica directamente las cuatro combinaciones de la tabla de or.
print("False or False:", False or False)
print("False or True :", False or True)
print("True  or False:", True or False)
print("True  or True :", True or True)

# Ejemplo aplicado: para aprobar se requieren nota y asistencia mínimas.
nota = 4.2
asistencia = 0.90
cumple_nota = nota >= 3.0
cumple_asistencia = asistencia >= 0.80
aprobo = cumple_nota and cumple_asistencia

print("Cumple el requisito de nota:", cumple_nota)
print("Cumple el requisito de asistencia:", cumple_asistencia)
print("¿Aprobó ambos requisitos?:", aprobo)
print("¿Incumplió algún requisito?:", not aprobo)
```

```
¿5 es igual a 5?: True
¿5 es diferente de 4?: True
¿El número 5 es igual al texto '5'?: False
False and False: False
False and True : False
True  and False: False
True  and True : True
False or False: False
False or True : True
True  or False: True
True  or True : True
Cumple el requisito de nota: True
Cumple el requisito de asistencia: True
¿Aprobó ambos requisitos?: True
¿Incumplió algún requisito?: False
```

---

4. Tipos de datos compuestos

↑ Volver al índice

Los tipos de datos compuestos permiten reunir varios valores dentro de un mismo objeto. En esta sección estudiaremos tres estructuras incorporadas en Python: listas, tuplas y diccionarios.

4.1 Mutabilidad e inmutabilidad

Un objeto es **mutable** cuando su contenido puede cambiar después de haber sido creado, sin necesidad de reemplazarlo por otro objeto. Por ejemplo, podemos agregar, eliminar o sustituir elementos de una lista y agregar o modificar pares clave-valor de un diccionario. Por ello, las **listas (`list`) y los diccionarios (`dict`) son mutables**. Esta propiedad es útil cuando la información debe actualizarse, pero también exige cuidado: dos variables pueden referirse al mismo objeto mutable y observar los mismos cambios.

Un objeto es **inmutable** cuando su contenido no puede modificarse después de su creación. Las **tuplas (`tuple`) son inmutables**: no es posible reemplazar, agregar ni eliminar directamente sus elementos. Si parece que una tupla fue modificada, en realidad se creó una nueva y la variable pasó a referirse a ella. Esta característica resulta útil para representar colecciones que no deberían cambiar. Como precisión, una tupla puede contener un objeto mutable —por ejemplo, una lista—; lo que no puede cambiarse es la estructura de la propia tupla.

4.2 Indexación y *slicing*

La **indexación** permite localizar y recuperar un elemento particular de una colección. En las secuencias ordenadas, como listas y tuplas, se escribe `secuencia[indice]`. **En Python la indexación empieza en `0` (como en cualquier lenguaje de programación serio :D)**: el primer elemento tiene índice `0`, el segundo índice `1`, y así sucesivamente. Los índices negativos cuentan desde el final: `-1` identifica el último elemento y `-2`, el penúltimo.

El ***slicing*** o corte extrae una porción mediante `secuencia[inicio:fin:paso]`. El índice `inicio` se incluye, pero `fin` se excluye; cualquiera de los tres componentes puede omitirse. Por ejemplo, `datos[1:4]` obtiene las posiciones `1`, `2` y `3`, mientras que `datos[::2]` toma un elemento de cada dos.

Los diccionarios funcionan de otra manera: no se consultan por posición, sino por **clave**, con `diccionario[clave]`. Por ejemplo, `persona["nombre"]` devuelve el valor asociado con la clave `"nombre"`. Cada clave debe ser única e inmutable (por ejemplo, un texto, un número o una tupla adecuada).

> **Atributos, métodos y operaciones.** `list`, `tuple` y `dict` no suelen ofrecer atributos públicos de datos como `.shape` en un arreglo de NumPy. Para conocer su tipo o tamaño usamos las funciones `type(objeto)` y `len(objeto)`; para consultar su contenido empleamos índices, cortes o claves. Sus comportamientos se exponen principalmente mediante **métodos**, que se llaman con la sintaxis `objeto.metodo(...)`. Técnicamente, un método también es un atributo del objeto, pero es un atributo invocable.

4.3 Listas (`list`)

Una lista es una colección **ordenada y mutable**, delimitada por corchetes: `lista = [elemento_1, elemento_2, ...]`. Puede contener valores de distintos tipos, conservar elementos repetidos y crecer o reducirse.

Operaciones y métodos importantes:

- `len(lista)`, `elemento in lista`, `lista[i]` y `lista[inicio:fin]`: consultan la longitud, la pertenencia, un elemento o una porción.
- `.append(x)`: agrega un elemento al final.
- `.extend(iterable)`: agrega todos los elementos de otra colección.

- `.insert(i, x)`: inserta `x` en la posición `i`.
- `.remove(x)`: elimina la primera aparición de `x`; produce un error si no existe.
- `.pop(i)`: elimina y devuelve el elemento de la posición `i`; sin argumento, usa el último.
- `.index(x)` y `.count(x)`: encuentran la primera posición de `x` y cuentan sus apariciones.
- `.sort()` y `.reverse()`: ordenan e invierten la lista **en el mismo objeto**.
- `.copy()`: crea una copia superficial independiente.

> Métodos como `.append()`, `.extend()`, `.insert()`, `.remove()`, `.sort()` y `.reverse()` modifican la lista y normalmente devuelven `None`; por eso se ejecutan primero y se imprime la lista después.

```
# Crea una lista mutable que conserva el orden y admite repetidos.
frutas = ["manzana", "pera", "mango", "pera"]

# Consultas: tipo, longitud, indexación, slicing y pertenencia
print("Lista original:", frutas)
print("Tipo:", type(frutas))
print("Longitud:", len(frutas))
print("Primer elemento [0]:", frutas[0])
print("Último elemento [-1]:", frutas[-1])
print("Porción [1:3]:", frutas[1:3])
print("¿Contiene 'mango'?:", "mango" in frutas)

# Métodos que agregan elementos y modifican la misma lista
frutas.append("uva")
frutas.extend(["banano", "fresa"])
frutas.insert(1, "naranja")
print("Después de agregar elementos:", frutas)

# Métodos de búsqueda
print("Índice de 'mango':", frutas.index("mango"))
print("Número de peras:", frutas.count("pera"))

# Métodos que eliminan elementos
fruta_eliminada = frutas.pop()
print("Elemento retirado con pop():", fruta_eliminada)
frutas.remove("pera")
print("Después de remove():", frutas)

# copy() permite ordenar una copia sin alterar la lista original
copia_frutas = frutas.copy()
copia_frutas.sort()
print("Copia ordenada:", copia_frutas)
print("Lista original sin ordenar:", frutas)
```

```
Lista original: ['manzana', 'pera', 'mango', 'pera']
Tipo: <class 'list'>
Longitud: 4
Primer elemento [0]: manzana
Último elemento [-1]: pera
Porción [1:3]: ['pera', 'mango']
¿Contiene 'mango'?: True
Después de agregar elementos: ['manzana', 'naranja', 'pera', 'mango', 'pera', 'uva', 'banano', 'fresa']
Índice de 'mango': 3
Número de peras: 2
Elemento retirado con pop(): fresa
Después de remove(): ['manzana', 'naranja', 'mango', 'pera', 'uva', 'banano']
Copia ordenada: ['banano', 'mango', 'manzana', 'naranja', 'pera', 'uva']
Lista original sin ordenar: ['manzana', 'naranja', 'mango', 'pera', 'uva', 'banano']
```

4.4 Tuplas (`tuple`)

Una tupla es una colección **ordenada e inmutable**, normalmente delimitada por paréntesis: `tupla = (elemento_1, elemento_2, ...)`. Puede contener tipos diferentes y valores repetidos. Para crear una tupla de un solo elemento es indispensable la coma: `(10,)`; `(10)` es simplemente el número `10`.

Como es inmutable, una tupla no tiene métodos como `.append()`, `.remove()` o `.sort()`. Sus principales operaciones y métodos son:

- `len(tupla)`, `elemento in tupla`, `tupla[i]` y `tupla[inicio:fin]`: consultan sin modificar.
- `.count(x)`: cuenta cuántas veces aparece `x`.
- `.index(x)`: devuelve el índice de la primera aparición de `x`; produce un error si no existe.
- **Desempaquetamiento**: asigna sus elementos a varias variables, como en `x, y = punto`.

```
# Crea una tupla de tres valores y otra de un solo elemento.
coordenadas = (4, 7, 4)
tupla_un_elemento = (10,)

# Consultas: tipo, longitud, indexación, slicing y pertenencia
print("Tupla:", coordenadas)
print("Tipo:", type(coordenadas))
print("Longitud:", len(coordenadas))
print("Primer elemento [0]:", coordenadas[0])
print("Último elemento [-1]:", coordenadas[-1])
print("Porción [0:2]:", coordenadas[0:2])
print("¿Contiene 7?:", 7 in coordenadas)

# Los dos métodos disponibles para consultar una tupla
print("Número de veces que aparece 4:", coordenadas.count(4))
print("Primera posición de 7:", coordenadas.index(7))
print("Tupla de un elemento:", tupla_un_elemento)

# Desempaquetamiento
x, y = (4, 7)
print("x =", x, "; y =", y)

# Esta instrucción produciría TypeError porque la tupla es inmutable:
# coordenadas[0] = 10
```

```
Tupla: (4, 7, 4)
Tipo: <class 'tuple'>
Longitud: 3
Primer elemento [0]: 4
Último elemento [-1]: 4
Porción [0:2]: (4, 7)
¿Contiene 7?: True
Número de veces que aparece 4: 2
Primera posición de 7: 1
Tupla de un elemento: (10,)
x = 4 ; y = 7
```

4.5 Diccionarios (`dict`)

Un diccionario es una colección **mutable** de pares `clave: valor`, delimitada por llaves: `diccionario = {"clave": valor}`. Cada clave es única; asignar otro valor a una clave existente actualiza esa entrada. Los diccionarios conservan el orden de inserción, pero se consultan por clave y no por un índice posicional.

Operaciones y métodos importantes:

- `diccionario[clave]`: devuelve el valor y produce `KeyError` si la clave no existe.
- `.get(clave, valor_predeterminado)`: consulta de forma segura y evita ese error.
- `.keys()`, `.values()` y `.items()`: presentan vistas de las claves, los valores y los pares, respectivamente.

- `.update(otro)`: agrega pares nuevos y actualiza las claves existentes.
- `.setdefault(clave, valor)`: devuelve el valor existente o crea la clave con el valor indicado.
- `.pop(clave)`: elimina la clave y devuelve su valor.
- `.copy()`: crea una copia superficial; `.clear()` elimina todos los pares.
- `len(diccionario)` y `clave in diccionario`: consultan el número de pares y la presencia de una clave.

```
# Relaciona las claves 'nombre' y 'edad' con sus valores iniciales.
persona = {"nombre": "Ana", "edad": 20}

# Consultas por clave y consulta segura con get()
print("Diccionario original:", persona)
print("Tipo:", type(persona))
print("Número de pares:", len(persona))
print("Nombre:", persona["nombre"])
print("Ciudad:", persona.get("ciudad", "No registrada"))
print("¿Existe la clave 'edad'?:", "edad" in persona)

# Vistas del contenido
print("Claves:", list(persona.keys()))
print("Valores:", list(persona.values()))
print("Pares:", list(persona.items()))

# Asignar y actualizar modifica el mismo diccionario
persona["ciudad"] = "Bogotá"
persona["edad"] = 21
persona.update({"programa": "Economía", "es_estudiante": True})
persona.setdefault("país", "Colombia")
print("Diccionario actualizado:", persona)

# pop() elimina una clave y devuelve su valor
edad_eliminada = persona.pop("edad")
print("Edad retirada:", edad_eliminada)
print("Diccionario después de pop():", persona)

# copy() crea otro diccionario con el mismo contenido
copia_persona = persona.copy()
print("Copia:", copia_persona)
```

```
Diccionario original: {'nombre': 'Ana', 'edad': 20}
Tipo: <class 'dict'>
Número de pares: 2
Nombre: Ana
Ciudad: No registrada
¿Existe la clave 'edad'?: True
Claves: ['nombre', 'edad']
Valores: ['Ana', 20]
Pares: [('nombre', 'Ana'), ('edad', 20)]
Diccionario actualizado: {'nombre': 'Ana', 'edad': 21, 'ciudad': 'Bogotá', 'programa': 'Economía',
'es_estudiante': True, 'país': 'Colombia'}
Edad retirada: 21
Diccionario después de pop(): {'nombre': 'Ana', 'ciudad': 'Bogotá', 'programa': 'Economía', 'es_estudiante':
True, 'país': 'Colombia'}
Copia: {'nombre': 'Ana', 'ciudad': 'Bogotá', 'programa': 'Economía', 'es_estudiante': True, 'país': 'Colombia'}
```

Regla fundamental de la sintaxis de Python

Todo encabezado de un **condicional** (`if`, `elif`, `else`), un **ciclo** (`for`, `while`), una **función** definida con `def` o una **clase** definida con `class` debe terminar con dos puntos (`:`). Las instrucciones que pertenecen a ese encabezado deben escribirse debajo y con sangría.

```
# Estructura condicional if-elif-else
if condicion:
    instruccion
elif otra_condicion:
    otra_instruccion
else:
    instruccion_alternativa
```

```
# Ciclo for
for elemento in coleccion:
    instruccion

# Ciclo while
while condicion:
    instruccion
    actualizar_condicion

# Función definida con def
def nombre_funcion():
    instruccion

# Clase definida con class
class NombreClase:
    instruccion
```

En Python, la **sangría o indentación es parte de la sintaxis**: delimita los bloques de código y determina qué instrucciones se ejecutan dentro de una condición, ciclo, función o clase. Una sangría incorrecta puede producir un error o, incluso, cambiar la lógica del programa. La convención habitual es usar **cuatro espacios por nivel** y evitar mezclar tabulaciones con espacios.

En programación en general, una indentación consistente hace visible la estructura del programa, facilita su lectura y revisión, reduce errores y ayuda a que otras personas puedan mantener el código, incluso en lenguajes donde los bloques se delimitan con llaves.

---

5. Estructuras de control

↑ Volver al índice

Las estructuras de control permiten tomar decisiones y repetir instrucciones. La sangría (los espacios al inicio de una línea) es obligatoria en Python.

5.1 Condicionales: `if`, `elif` y `else`

La sintaxis básica de una estructura condicional es:

```
# Evalúa las condiciones en orden y ejecuta un solo bloque.
if condicion_1:
    instrucciones
elif condicion_2:
    otras_instrucciones
else:
    instrucciones_alternativas
```

Cada expresión ubicada después de `if` o `elif` es una **condición lógica**. Las comparaciones, como `nota >= 3.0`, devuelven directamente un booleano (`bool`): `True` o `False`. Si la condición es `True`, Python ejecuta el bloque con sangría situado debajo; si es `False`, omite ese bloque y continúa con el siguiente `elif` o, si corresponde, con `else`. Cada encabezado termina con dos puntos y cada bloque debe conservar la indentación.

Python evalúa las condiciones de arriba hacia abajo y ejecuta solamente el primer bloque cuya condición sea verdadera. Después salta al código posterior a toda la estructura. Si todas son falsas, ejecuta `else` cuando este existe; si no hay `else`, continúa directamente con la siguiente instrucción del programa.

> **Precisión:** una condición debe poder interpretarse como verdadera o falsa, pero no está obligada a ser literalmente un objeto `bool`. Python aplica internamente una prueba equivalente a `bool(condicion)`. Por ejemplo, `0`, `None`, `""` y las colecciones vacías se interpretan como `False`; los números distintos de cero y las colecciones no vacías suelen interpretarse como `True`. Para comenzar, es recomendable escribir comparaciones explícitas que produzcan un booleano fácil de leer.

```
# Define la calificación que evaluará la estructura condicional.
nota = 4.0

# Ejecuta únicamente el primer bloque cuya condición sea verdadera.
if nota >= 4.5:
    print("Excelente")
elif nota >= 3.0:
    print("Aprobaste")
else:
    print("No aprobaste")
```

```
Aprobaste
```

5.2 Ciclo `for`

Antes de comparar los ciclos, conviene distinguir dos variables frecuentes:

- Una **variable contadora** registra cuántas iteraciones han ocurrido o cuál posición se está procesando. Suele comenzar en `0` y aumentar en una unidad con `contador += 1`. En un `for` que recorre directamente una colección, Python controla internamente el avance y normalmente no necesitamos escribir un contador.
- Una **variable acumuladora** conserva un resultado parcial y lo actualiza en cada iteración. Para una suma suele comenzar en `0`, el elemento neutro de la adición; para un producto comenzaría normalmente en `1`. Debe inicializarse **antes** del ciclo para no perder lo acumulado en iteraciones anteriores.

El contador responde «¿por cuál iteración o posición voy?», mientras que el acumulador responde «¿qué resultado he reunido hasta ahora?». En el siguiente problema elevaremos cada número al cuadrado y agregaremos ese cuadrado al acumulador.

La sintaxis básica de un ciclo `for` es:

```
# Inicializa el acumulador antes de comenzar el recorrido.
acumulador = 0

# Repite el bloque una vez por cada elemento de la colección.
for elemento in coleccion:
    acumulador += elemento
```

En cada iteración, `elemento` toma directamente el siguiente valor de `coleccion`. La línea del `for` termina con dos puntos y las instrucciones que se repiten deben llevar sangría.

Este es el enfoque más directo cuando solo necesitamos los valores y no sus posiciones.

```
# Define la lista y el acumulador de la suma de cuadrados.
numeros = [2, 3, 5, 7]
suma_cuadrados_for = 0

# El for entrega directamente un elemento en cada iteración.
for numero in numeros:
    cuadrado = numero ** 2
    suma_cuadrados_for += cuadrado
    print(f"{numero} al cuadrado es {cuadrado}")
```

```
# El resultado queda almacenado al terminar el ciclo.
print("Suma de cuadrados con for directo:", suma_cuadrados_for)
```

```
2 al cuadrado es 4
3 al cuadrado es 9
5 al cuadrado es 25
7 al cuadrado es 49
Suma de cuadrados con for directo: 87
```

5.2.1 Repeticiones con `range()`

La sintaxis general de `range()` dentro de un ciclo es:

```
# Genera los índices válidos desde 0 hasta len(coleccion) - 1.
for indice in range(len(coleccion)):
    elemento = coleccion[indice]
    instrucciones
```

`range(inicio, fin, paso)` genera una secuencia de enteros: `inicio` es el primer valor, `fin` no se incluye y `paso` indica cuánto cambia el número. `inicio` y `paso` pueden omitirse; sus valores predeterminados son `0` y `1`.

Para recorrer índices usamos `range(len(coleccion))`: `len()` devuelve la cantidad de elementos y `range()` produce desde `0` hasta el último índice válido. Este enfoque es útil cuando la posición importa, aunque resulta más extenso que recorrer directamente los elementos.

```
# Define la misma lista y reinicia el acumulador en cero.
numeros = [2, 3, 5, 7]
suma_cuadrados_range = 0

# range(len(numeros)) produce los índices 0, 1, 2 y 3.
for indice in range(len(numeros)):
    numero = numeros[indice]
    cuadrado = numero ** 2
    suma_cuadrados_range += cuadrado
    print(f"Índice {indice}: {numero} al cuadrado es {cuadrado}")

# Muestra la suma obtenida después de visitar todos los índices.
print("Suma de cuadrados con for y range:", suma_cuadrados_range)
```

```
Índice 0: 2 al cuadrado es 4
Índice 1: 3 al cuadrado es 9
Índice 2: 5 al cuadrado es 25
Índice 3: 7 al cuadrado es 49
Suma de cuadrados con for y range: 87
```

5.3 Ciclo `while`

La sintaxis básica de un ciclo `while` es:

```
# Inicializa explícitamente el índice contador y el acumulador.
indice = 0
acumulador = 0

# Repite el bloque mientras el índice permanezca dentro de la colección.
while indice < len(coleccion):
    acumulador += coleccion[indice]
    indice += 1
```

La expresión escrita después de `while` también es una **condición lógica**. Se evalúa antes de cada iteración y debe producir o poder interpretarse como un valor booleano. Si es `True`, Python ejecuta el bloque con sangría y vuelve a evaluar la condición; si es `False`, omite el bloque, termina el ciclo y salta a la primera instrucción situada después del `while`.

En `while indice < len(coleccion)`, la comparación `<` devuelve literalmente un `bool`. Al igual que en `if`, Python también acepta objetos con valor de verdad y aplica una prueba equivalente a `bool(condicion)`, aunque una comparación explícita suele comunicar mejor la regla de continuación. A diferencia de `for`, `while` no avanza automáticamente: debemos inicializar el índice, comprobarlo y actualizarlo. Si olvidamos `indice += 1`, la condición puede permanecer verdadera y producir un ciclo infinito.

Este control manual resulta útil cuando no conocemos anticipadamente el número de iteraciones o cuando la repetición depende de una condición más general. Para recorrer una lista completa, suele ser menos conciso que `for`.

```
# Define la misma lista, el índice contador y el acumulador.
numeros = [2, 3, 5, 7]
indice = 0
suma_cuadrados_while = 0

# Usa el índice para consultar un elemento mientras queden posiciones.
while indice < len(numeros):
    numero = numeros[indice]
    cuadrado = numero ** 2
    suma_cuadrados_while += cuadrado
    print(f"Índice {indice}: {numero} al cuadrado es {cuadrado}")

    # Incrementa el contador después de procesar la posición actual.
    indice += 1

# Muestra la suma obtenida al volver falsa la condición del while.
print("Suma de cuadrados con while:", suma_cuadrados_while)
```

```
Índice 0: 2 al cuadrado es 4
Índice 1: 3 al cuadrado es 9
Índice 2: 5 al cuadrado es 25
Índice 3: 7 al cuadrado es 49
Suma de cuadrados con while: 87
```

5.4 Comparación de los tres enfoques

Los tres ciclos producen el mismo resultado porque parten del acumulador `0`, visitan exactamente una vez los mismos cuatro elementos, calculan `numero ** 2` y agregan cada cuadrado. El mecanismo de recorrido cambia, pero no cambia la operación matemática:

$$2^2 + 3^2 + 5^2 + 7^2 = 4 + 9 + 25 + 49 = 87.$$

Enfoque	Valor disponible en cada iteración	Control del índice	Ventaja principal	Riesgo o costo
`for numero in numeros`	El elemento directamente	Implícito, administrado por Python	Es el más claro y conciso cuando solo interesa el valor	No entrega la posición de manera directa
`for indice in range(len(numeros))`	El índice; el valor se obtiene con `numeros[indice]`	Explícito mediante `range()`	Resulta útil cuando también se necesita la posición	Es más extenso y puede facilitar errores de índices
`while indice < len(numeros)`	El índice y el valor consultado manualmente	Totalmente manual	Permite condiciones de terminación flexibles	Olvidar actualizar el contador puede crear un ciclo infinito

En los tres casos, el **acumulador es explícito** y contiene las sumas parciales. El `for` directo no necesita un contador escrito por nosotros; `range()` expone el índice como variable del ciclo; y `while` requiere inicializarlo y aumentarlo manualmente. Los tres realizan un número de operaciones proporcional a la longitud de la lista, es decir, tienen complejidad temporal $O(n)$ y requieren solamente $O(1)$ memoria adicional.

> Como todavía no estamos encapsulando el procedimiento en una función, aquí «retornar el resultado» significa dejar la suma final almacenada en una variable y mostrarla con `print()`. En la sección 6 veremos cómo una función puede entregarla formalmente mediante `return`.

---

6. Funciones con def

↑ Volver al índice

Sintaxis básica de una función

La palabra reservada `def` inicia la definición de una función. Después se escribe su nombre, los parámetros entre paréntesis y dos puntos. El cuerpo debe llevar sangría, y `return` permite devolver un resultado; si se omite, la función devuelve `None`.

Cómo escribir un buen docstring de función

Un **docstring** es la cadena de documentación situada inmediatamente después del encabezado `def`, delimitada habitualmente por comillas triples (`"""`). A diferencia de un comentario con `#`, que explica decisiones internas a quien lee el código, el docstring describe la **interfaz pública** de la función: qué hace, qué recibe, qué devuelve y qué errores puede producir. Python lo guarda en `funcion.__doc__` y herramientas como `help(funcion)` pueden mostrarlo.

Seguiremos PEP 257 y la convención de NumPy/SciPy: una primera línea breve, clara y en modo imperativo; una línea en blanco; y solamente las secciones pertinentes. `Parameters` documenta nombre, tipo y significado de cada parámetro; `Returns` describe el tipo y significado del resultado; y `Raises` enumera excepciones importantes. No se debe repetir literalmente el código ni documentar parámetros inexistentes.

```
def sumar(numero_1, numero_2):
    """Suma dos números.

    Parameters
    ----------
    numero_1 : int or float
        Primer sumando.
    numero_2 : int or float
        Segundo sumando.

    Returns
    -------
    int or float
        Suma de los dos números.
    """
    # Calcula el valor que la función entregará al llamador.
    resultado = numero_1 + numero_2
    return resultado

# Llama la función y conserva el valor retornado.
valor = sumar(4, 6)
```

Una función definida con `def` agrupa instrucciones que podemos reutilizar. Implementaremos con funciones las mismas tareas econométricas que luego resolveremos con la clase `MCO`: validar y mostrar los datos, describir y estimar el modelo, imprimir sus resultados y probar hipótesis.

6.1 ¿Por qué es programación imperativa?

La **programación imperativa** describe paso a paso cómo debe realizarse una tarea mediante una secuencia de instrucciones. Cuando esas instrucciones se organizan en funciones también se habla de un estilo **procedimental**. En el ejemplo, primero cargamos los datos, después estimamos el modelo, guardamos el resultado en una variable y finalmente imprimimos resultados y ejecutamos una prueba. El orden de estas operaciones es explícito y cada paso puede cambiar el estado del programa al crear o modificar variables.

En este enfoque, los datos, la fórmula y los resultados existen por separado y deben pasarse como argumentos a las funciones que los necesitan. A diferencia de la programación orientada a objetos de la sección 7, no creamos un objeto que recuerde internamente el estado de la estimación. El enfoque imperativo suele ser directo y apropiado para procedimientos cortos; el enfoque orientado a objetos resulta útil cuando queremos reunir estado y comportamiento en una unidad reutilizable.

Para construir un equivalente del contador compartido de la clase `MCO`, usaremos la variable global `estimaciones_realizadas`. La instrucción `global estimaciones_realizadas` permite que `estimar_mco()` modifique esa variable externa. Este recurso hace visible el estado compartido, pero puede volverse difícil de controlar en programas grandes; en la sección 7, ese estado queda dentro de la clase como `MCO.modelos_creados`.

6.2 Definición de las funciones

```
# Importa las librerías usadas para validar datos y estimar por fórmulas.
import pandas as pd
# Importa la interfaz de fórmulas de statsmodels para estimar el MCO.
import statsmodels.formula.api as smf

# Lleva un conteo global de las estimaciones realizadas.
estimaciones_realizadas = 0


def validar_datos(datos):
    """Valida que los datos sean un DataFrame.

    Parameters
    ----------
    datos : pandas.DataFrame
        Datos que se utilizarán en el análisis.

    Raises
    ------
    TypeError
        Si `datos` no es un DataFrame de pandas.
    """
    # Detiene el procedimiento temprano si el objeto no tiene el tipo esperado.
    if not isinstance(datos, pd.DataFrame):
        raise TypeError("datos debe ser un DataFrame de pandas.")


def describir_modelo(datos, formula, cov_type="nonrobust", resultados=None):
    """Muestra la configuración y el estado de un modelo MCO.

    Parameters
    ----------
    datos : pandas.DataFrame
        Datos utilizados por el modelo.
    formula : str
        Fórmula compatible con Patsy, por ejemplo `y ~ x1 + x2`.
    cov_type : str, default='nonrobust'
        Tipo de estimador para la matriz de covarianzas.
    resultados : object, optional
        Resultados ajustados; `None` indica que aún no se ha estimado.
    """
    # Verifica los datos antes de construir la descripción.
    validar_datos(datos)
```

```
    # Determina el estado a partir de la presencia de resultados.
    estado = "estimado" if resultados is not None else "sin estimar"
    print(
        f"MCO(formula='{formula}', observaciones={len(datos)}, "
        f"cov_type='{cov_type}', estado='{estado}')"
    )


def mostrar_datos(datos, n=5):
    """Muestra las primeras filas de un DataFrame.

    Parameters
    ----------
    datos : pandas.DataFrame
        Datos que se desean inspeccionar.
    n : int, default=5
        Número de filas que se mostrarán.
    """
    # Valida el objeto antes de llamar el método head().
    validar_datos(datos)
    print(datos.head(n))


def estimar_mco(datos, formula, cov_type="nonrobust"):
    """Estima un modelo lineal mediante mínimos cuadrados ordinarios.

    Parameters
    ----------
    datos : pandas.DataFrame
        Datos que contienen las variables de la fórmula.
    formula : str
        Especificación del modelo en la sintaxis de Patsy.
    cov_type : str, default='nonrobust'
        Tipo de estimador para la matriz de covarianzas.

    Returns
    -------
    statsmodels.regression.linear_model.RegressionResultsWrapper
        Resultados del modelo estimado.

    Raises
    ------
    TypeError
        Si `datos` no es un DataFrame de pandas.
    """
    # Permite reasignar el contador definido en el alcance global.
    global estimaciones_realizadas
    validar_datos(datos)
    # Define el modelo, lo ajusta y registra la estimación.
    modelo = smf.ols(formula=formula, data=datos)
    resultados = modelo.fit(cov_type=cov_type)
    estimaciones_realizadas += 1
    return resultados


def mostrar_estimaciones_realizadas():
    """Muestra el número acumulado de estimaciones MCO."""
    # Lee el contador global sin modificarlo.
    print(f"Estimaciones MCO realizadas: {estimaciones_realizadas}")


def mostrar_resultados(resultados):
    """Muestra el resumen estadístico de un modelo estimado.

    Parameters
    ----------
    resultados : statsmodels result object
        Objeto que proporciona el método `summary()`.
    """
    # summary() genera la tabla completa de resultados.
    print(resultados.summary())


def probar_hipotesis(resultados, restriccion):
```

```
    """Ejecuta y muestra una prueba t de una restricción lineal.

    Parameters
    ----------
    resultados : statsmodels result object
        Resultados de un modelo previamente estimado.
    restriccion : str
        Hipótesis escrita en la sintaxis aceptada por `t_test()`.

    Returns
    -------
    statsmodels.stats.contrast.ContrastResults
        Resultado de la prueba de hipótesis.
    """
    # Calcula la prueba, la presenta y devuelve el objeto para reutilizarlo.
    prueba = resultados.t_test(restriccion)
    print(prueba)
    return prueba
```

6.3 Ejemplo aplicado

Aplicaremos las funciones al conjunto **`longley`**. Las llamadas aparecen en el orden en que se ejecuta el procedimiento, y el resultado de la estimación debe guardarse explícitamente en **`resultados_empleo`**.

```
# Importa statsmodels para acceder al conjunto de datos longley.
import statsmodels.api as sm

# Carga los datos y define la especificación común de la estimación.
datos_longley_funciones = sm.datasets.longley.load_pandas().data
formula_empleo = "TOTEMP ~ GNP + UNEMP"
tipo_covarianza = "HC1"

# Describe el modelo aún no estimado y revisa las primeras observaciones.
describir_modelo(
    datos_longley_funciones, formula_empleo, tipo_covarianza
)
mostrar_datos(datos_longley_funciones)

# Estima el modelo y conserva explícitamente el objeto de resultados.
resultados_empleo = estimar_mco(
    datos_longley_funciones, formula_empleo, tipo_covarianza
)

# Presenta el nuevo estado y el resumen estadístico de la estimación.
describir_modelo(
    datos_longley_funciones,
    formula_empleo,
    tipo_covarianza,
    resultados_empleo,
)
mostrar_estimaciones_realizadas()
mostrar_resultados(resultados_empleo)

# Contrasta la hipótesis nula de que el coeficiente de GNP es cero.
prueba_gnp_funciones = probar_hipotesis(
    resultados_empleo, "GNP = 0"
)
```

```
MCO(formula='TOTEMP ~ GNP + UNEMP', observaciones=16, cov_type='HC1', estado='sin estimar')
    TOTEMP  GNPDEFL       GNP   UNEMP   ARMED       POP    YEAR
0  60323.0     83.0  234289.0  2356.0  1590.0  107608.0  1947.0
1  61122.0     88.5  259426.0  2325.0  1456.0  108632.0  1948.0
2  60171.0     88.2  258054.0  3682.0  1616.0  109773.0  1949.0
3  61187.0     89.5  284599.0  3351.0  1650.0  110929.0  1950.0
4  63221.0     96.2  328975.0  2099.0  3099.0  112075.0  1951.0
MCO(formula='TOTEMP ~ GNP + UNEMP', observaciones=16, cov_type='HC1', estado='estimado')
Estimaciones MCO realizadas: 1
                            OLS Regression Results
==============================================================================
Dep. Variable:                 TOTEMP   R-squared:                       0.981
Model:                            OLS   Adj. R-squared:                  0.978
```

```
Method:                 Least Squares   F-statistic:                     503.9
Date:                Sun, 13 Sep 2026   Prob (F-statistic):           4.81e-13
Time:                        12:13:56   Log-Likelihood:                -121.25
No. Observations:                  16   AIC:                             248.5
Df Residuals:                      13   BIC:                             250.8
Df Model:                           2
Covariance Type:                  HC1
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
Intercept   5.238e+04    471.519    111.092      0.000    5.15e+04    5.33e+04
GNP            0.0378      0.001     25.722      0.000       0.035       0.041
UNEMP         -0.5436      0.165     -3.288      0.001      -0.868      -0.220
==============================================================================
Omnibus:                        1.922   Durbin-Watson:                   0.976
Prob(Omnibus):                  0.382   Jarque-Bera (JB):                0.677
Skew:                           0.483   Prob(JB):                        0.713
Kurtosis:                       3.288   Cond. No.                     1.75e+06
==============================================================================

Notes:
[1] Standard Errors are heteroscedasticity robust (HC1)
[2] The condition number is large, 1.75e+06. This might indicate that there are
strong multicollinearity or other numerical problems.
                            Test for Constraints
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
c0             0.0378      0.001     25.722      0.000       0.035       0.041
==============================================================================
```

---

7. Clases con `class`

↑ Volver al índice

Sintaxis básica de una clase

La palabra reservada de Python es `class`, escrita en minúscula. A continuación se indica el nombre de la clase —por convención en formato `CamelCase`— y dos puntos. Su cuerpo lleva sangría y puede contener atributos y métodos definidos con `def`. El parámetro `self` representa la instancia sobre la que trabaja cada método.

Cómo escribir un buen docstring de clase

El docstring de una clase se escribe como su primera instrucción y explica **qué representa la clase y cuál es su responsabilidad**. Siguiendo PEP 257 y el formato de NumPy/SciPy, puede incluir `Parameters` para los argumentos usados al construir una instancia y `Attributes` para los datos públicos que el usuario podrá consultar. Los métodos públicos deben tener sus propios docstrings con `Parameters`, `Returns` y `Raises` cuando corresponda. No es necesario incluir `self` ni `cls` en `Parameters`, porque Python los suministra automáticamente.

Un buen docstring complementa los comentarios: documenta cómo usar la clase; los comentarios con `#` aclaran decisiones o pasos internos de la implementación. Puede consultarse con `help(NombreClase)` o mediante `NombreClase.__doc__`.

```
class Estudiante:
    """Representa a un estudiante y su calificación.

    Parameters
    ----------
    nombre : str
        Nombre del estudiante.
    nota : float
        Calificación observada.
```

```
    Attributes
    ----------
    nombre : str
        Nombre almacenado en la instancia.
    nota : float
        Calificación almacenada en la instancia.
    """

    def __init__(self, nombre, nota):
        """Inicializa un estudiante con su nombre y nota."""
        # Cada asignación crea un atributo propio de la instancia.
        self.nombre = nombre
        self.nota = nota

    def mostrar_nota(self):
        """Muestra la calificación del estudiante."""
        print(self.nota)

# Crea una instancia y llama uno de sus métodos públicos.
estudiante = Estudiante("Ana", 4.5)
estudiante.mostrar_nota()
```

Diferencia entre una clase y un objeto

Una **clase** es la definición o el molde que establece qué datos y comportamientos tendrá un determinado tipo de elemento. Una **instancia de clase**, también llamada **objeto**, es un elemento concreto creado a partir de ese molde. Por ejemplo, `MCO` es una clase, mientras que `modelo_empleo = MCO(...)` crea un objeto particular de esa clase. Es posible crear varias instancias de una misma clase y cada una puede conservar valores diferentes.

Un objeto puede contener **atributos** y **métodos**:

- Un **atributo** es un dato asociado al objeto y se consulta mediante `objeto.atributo`; por ejemplo, `modelo_empleo.formula`.
- Un **método** es una función asociada al objeto y se ejecuta mediante `objeto.metodo()`; por ejemplo, `modelo_empleo.estimar()`. Los paréntesis indican que el método se está llamando.

```
objeto.atributo       # Consultar un atributo
objeto.metodo()       # Ejecutar un método
```

Un atributo también puede almacenar **otro objeto**. Esta relación permite construir objetos complejos a partir de otros objetos y acceder a sus elementos encadenando puntos:

```
# Encadena atributos para consultar el objeto almacenado internamente.
objeto.objeto_interno.atributo
objeto.objeto_interno.metodo()
```

En nuestro ejemplo, después de estimar, `modelo_empleo.resultados` contiene un objeto de resultados creado por statsmodels. Por ello podemos consultar un atributo anidado con `modelo_empleo.resultados.params` o ejecutar uno de sus métodos con `modelo_empleo.resultados.summary()`. Si el atributo interno se llamara `argumento`, la sintaxis sería `objeto.objeto_interno.argumento`; sin embargo, conviene recordar que un **argumento** es originalmente un valor que se entrega a una función o método, mientras que un **atributo** es un valor almacenado en un objeto.

Una clase permite reunir datos y operaciones relacionadas dentro de un mismo objeto. Construiremos una clase llamada `MCO` para representar el modelo de mínimos cuadrados ordinarios

$$y_i = \beta_0 + \beta_1 x_{1i} + \cdots + \beta_k x_{ki} + u_i.$$

7.1 ¿Por qué es programación orientada a objetos?

La **programación orientada a objetos (POO)** organiza el programa alrededor de objetos que combinan **estado** y **comportamiento**. La clase `MCO` funciona como un molde y cada instancia contiene sus propios datos, fórmula, tipo de covarianza y resultados. Sus métodos actúan sobre ese estado; por ejemplo, `modelo_empleo.estimar()` guarda la estimación dentro de `modelo_empleo.resultados`.

El contraste con la sección 6 está en dónde se conserva la información. En el enfoque imperativo debemos pasar `datos_longley_funciones` y `resultados_empleo` de una función a otra. En el enfoque orientado a objetos basta con enviar los datos al crear la instancia y luego pedirle al mismo objeto que muestre datos, estime o pruebe una hipótesis. El primer enfoque hace visible la secuencia completa; el segundo encapsula el estado y ofrece una interfaz común para trabajar con él.

7.2 Métodos especiales `__init__` y `__str__`

Los nombres `__init__` y `__str__` comienzan y terminan con dos guiones bajos; por eso se conocen como métodos *dunder* (*double underscore*). Son **especiales** porque forman parte del modelo de datos de Python y el propio lenguaje los ejecuta automáticamente en determinadas situaciones, sin que normalmente tengamos que llamarlos de manera directa:

- `__init__(self, ...)` **inicializa** una instancia que acaba de ser creada. Python lo llama automáticamente al escribir, por ejemplo, `MCO(datos, formula)`. En nuestra clase, guarda el `DataFrame`, la fórmula, el tipo de covarianza y el estado inicial del modelo.
- `__str__(self)` devuelve una representación legible del objeto. Python lo utiliza cuando se ejecuta `str(objeto)` o `print(objeto)`; por eso `print(modelo_empleo)` muestra la fórmula, el número de observaciones y el estado de la estimación.

7.3 Variables y métodos de clase frente a los de instancia

La palabra reservada `class` abre la definición de una clase. Una **variable de clase** se declara directamente dentro de ese bloque y es compartida por todas las instancias; en el ejemplo será `modelos_creados`. Un **método de clase** se marca con `@classmethod` y, por convención, recibe `cls` como primer argumento: `cls` representa la clase y permite consultar o modificar sus variables compartidas.

En cambio, `self` representa una instancia concreta. Expresiones como `self.datos` o `self.resultados` son variables de instancia, por lo que cada objeto conserva sus propios valores. Los métodos de instancia también reciben `self` como primer argumento y operan sobre ese objeto particular. En resumen: `class` define la clase, `cls` se usa para la clase y `self` para cada instancia.

7.4 Definición de la clase

```
# Importa las librerías requeridas por la clase MCO.
import pandas as pd
import statsmodels.formula.api as smf


class MCO:
    """Representa un modelo lineal de mínimos cuadrados ordinarios.

    Parameters
    ----------
    datos : pandas.DataFrame
        Datos que contienen las variables del modelo.
    formula : str
        Especificación compatible con Patsy, como `y ~ x1 + x2`.
    cov_type : str, default='nonrobust'
        Tipo de estimador para la matriz de covarianzas.
```

```
    Attributes
    ----------
    modelos_creados : int
        Número de instancias creadas; es compartido por la clase.
    datos : pandas.DataFrame
        Copia de los datos asignada a la instancia.
    formula : str
        Fórmula almacenada en la instancia.
    cov_type : str
        Tipo de covarianza solicitado.
    resultados : object or None
        Resultados de statsmodels después de estimar; antes es `None`.
    """

    # Variable de clase compartida por todas las instancias MCO.
    modelos_creados = 0

    def __init__(self, datos, formula, cov_type="nonrobust"):
        """Inicializa la especificación y los datos de un modelo MCO.

        Raises
        ------
        TypeError
            Si `datos` no es un DataFrame de pandas.
        """
        # Valida el argumento antes de crear los atributos de instancia.
        if not isinstance(datos, pd.DataFrame):
            raise TypeError("datos debe ser un DataFrame de pandas.")

        # Copia los datos para aislar el estado interno del objeto.
        self.datos = datos.copy()
        self.formula = formula
        self.cov_type = cov_type
        self.resultados = None
        # Actualiza el contador compartido mediante la clase de la instancia.
        type(self).modelos_creados += 1

    def __str__(self):
        """Devuelve una representación legible del estado del modelo.

        Returns
        -------
        str
            Fórmula, observaciones, covarianza y estado de estimación.
        """
        # Traduce la presencia de resultados a una etiqueta comprensible.
        estado = "estimado" if self.resultados is not None else "sin estimar"
        return (
            f"MCO(formula='{self.formula}', observaciones={len(self.datos)}, "
            f"cov_type='{self.cov_type}', estado='{estado}')"
        )

    def mostrar_datos(self, n=5):
        """Muestra las primeras observaciones del modelo.

        Parameters
        ----------
        n : int, default=5
            Número de filas que se mostrarán.
        """
        # head() limita la vista sin modificar el DataFrame almacenado.
        print(self.datos.head(n))

    @classmethod
    def mostrar_modelos_creados(cls):
        """Muestra el número de objetos MCO creados."""
        # cls accede al estado compartido de la clase.
        print(f"Objetos MCO creados: {cls.modelos_creados}")

    def estimar(self):
        """Estima el modelo y almacena sus resultados.

        Returns
```

```
        -------
        statsmodels.regression.linear_model.RegressionResultsWrapper
            Resultados del modelo ajustado.
        """
        # Construye el modelo a partir de la fórmula y los datos del objeto.
        modelo = smf.ols(formula=self.formula, data=self.datos)
        self.resultados = modelo.fit(cov_type=self.cov_type)
        return self.resultados

    def mostrar_resultados(self):
        """Muestra el resumen del modelo previamente estimado.

        Raises
        ------
        RuntimeError
            Si todavía no se ha ejecutado `estimar()`.
        """
        # Impide consultar resultados que todavía no existen.
        self._verificar_estimacion()
        print(self.resultados.summary())

    def probar_hipotesis(self, restriccion):
        """Ejecuta y muestra una prueba t de una restricción lineal.

        Parameters
        ----------
        restriccion : str
            Hipótesis escrita en la sintaxis aceptada por `t_test()`.

        Returns
        -------
        statsmodels.stats.contrast.ContrastResults
            Resultado de la prueba de hipótesis.

        Raises
        ------
        RuntimeError
            Si el modelo aún no ha sido estimado.
        """
        # Verifica el estado antes de delegar la prueba a statsmodels.
        self._verificar_estimacion()
        prueba = self.resultados.t_test(restriccion)
        print(prueba)
        return prueba

    def _verificar_estimacion(self):
        """Verifica que la instancia contenga resultados estimados.

        Raises
        ------
        RuntimeError
            Si `resultados` todavía es `None`.
        """
        # Centraliza la precondición usada por los métodos de resultados.
        if self.resultados is None:
            raise RuntimeError("Primero debes ejecutar el método estimar().")
```

7.5 Ejemplo aplicado

Usaremos el conjunto de datos macroeconómicos **`longley`**, incluido en statsmodels y cargado como un **`DataFrame`**. Estimaremos el empleo total (**`TOTEMP`**) en función del producto nacional bruto (**`GNP`**) y el desempleo (**`UNEMP`**), con errores estándar robustos HC1. Finalmente, probaremos la hipótesis nula $H_0 : \beta_{GNP} = 0$.

```
# Importa statsmodels para cargar el conjunto macroeconómico longley.
import statsmodels.api as sm

# Obtiene el DataFrame que se entregará al constructor de MCO.
datos_longley = sm.datasets.longley.load_pandas().data

# Crea una instancia con su fórmula y covarianza robusta HC1.
modelo_empleo = MCO(
```

```
    datos=datos_longley,
    formula="TOTEMP ~ GNP + UNEMP",
    cov_type="HC1",
)

# Consulta la variable de clase y el estado inicial del objeto.
MCO.mostrar_modelos_creados()
print(modelo_empleo)
modelo_empleo.mostrar_datos()

# Estima el modelo; los resultados quedan almacenados en la instancia.
modelo_empleo.estimar()
print(modelo_empleo)
modelo_empleo.mostrar_resultados()

# Contrasta la hipótesis nula de que el coeficiente de GNP es cero.
prueba_gnp = modelo_empleo.probar_hipotesis("GNP = 0")
```

```
Objetos MCO creados: 1
MCO(formula='TOTEMP ~ GNP + UNEMP', observaciones=16, cov_type='HC1', estado='sin estimar')
    TOTEMP  GNPDEFL       GNP   UNEMP   ARMED       POP    YEAR
0  60323.0     83.0  234289.0  2356.0  1590.0  107608.0  1947.0
1  61122.0     88.5  259426.0  2325.0  1456.0  108632.0  1948.0
2  60171.0     88.2  258054.0  3682.0  1616.0  109773.0  1949.0
3  61187.0     89.5  284599.0  3351.0  1650.0  110929.0  1950.0
4  63221.0     96.2  328975.0  2099.0  3099.0  112075.0  1951.0
MCO(formula='TOTEMP ~ GNP + UNEMP', observaciones=16, cov_type='HC1', estado='estimado')
                            OLS Regression Results
==============================================================================
Dep. Variable:                 TOTEMP   R-squared:                       0.981
Model:                            OLS   Adj. R-squared:                  0.978
Method:                 Least Squares   F-statistic:                     503.9
Date:                Sun, 13 Sep 2026   Prob (F-statistic):           4.81e-13
Time:                        12:13:56   Log-Likelihood:                -121.25
No. Observations:                  16   AIC:                             248.5
Df Residuals:                      13   BIC:                             250.8
Df Model:                           2
Covariance Type:                  HC1
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
Intercept   5.238e+04    471.519    111.092      0.000    5.15e+04    5.33e+04
GNP            0.0378      0.001     25.722      0.000       0.035       0.041
UNEMP         -0.5436      0.165     -3.288      0.001      -0.868      -0.220
==============================================================================
Omnibus:                        1.922   Durbin-Watson:                   0.976
Prob(Omnibus):                  0.382   Jarque-Bera (JB):                0.677
Skew:                           0.483   Prob(JB):                        0.713
Kurtosis:                       3.288   Cond. No.                     1.75e+06
==============================================================================

Notes:
[1] Standard Errors are heteroscedasticity robust (HC1)
[2] The condition number is large, 1.75e+06. This might indicate that there are
strong multicollinearity or other numerical problems.
                           Test for Constraints
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
c0             0.0378      0.001     25.722      0.000       0.035       0.041
==============================================================================
```

8. Alcance de las variables (*scope*)

↑ Volver al índice

El **alcance** o ***scope*** es la región del programa desde la cual un nombre puede consultarse directamente. Cuando escribimos `precio`, Python debe determinar a qué objeto se refiere ese nombre. Para organizar esas asociaciones utiliza **espacios de nombres** (*namespaces*), que pueden imaginarse como tablas que relacionan nombres con objetos.

El alcance evita que todas las variables de un programa se mezclen. Dos funciones pueden usar una variable llamada `resultado` sin interferir entre sí, porque cada llamada crea su propio espacio local. También permite controlar qué información es temporal, cuál pertenece a un objeto y cuál debe compartirse.

8.1 La regla LEGB

Cuando Python encuentra un nombre sin prefijo, como `tasa`, lo busca siguiendo la regla **LEGB**, desde el alcance más cercano hacia el más amplio:

1. **L — Local:** nombres creados dentro de la función que se está ejecutando, incluidos sus parámetros. Cada llamada crea un nuevo alcance local.
2. **E — Enclosing (envolvente):** nombres de las funciones exteriores cuando una función está definida dentro de otra. Puede haber varios niveles envolventes.
3. **G — Global:** nombres definidos en el nivel principal del módulo. En un notebook corresponde normalmente al espacio del kernel compartido por las celdas ejecutadas.
4. **B — Built-in (incorporado):** nombres proporcionados por Python, como `print`, `len`, `sum` y `type`.

La búsqueda se detiene al encontrar la primera coincidencia. Por ello no conviene crear variables llamadas `list`, `sum` o `str`: ocultarían temporalmente las funciones incorporadas que tienen esos nombres.

> Un detalle importante: en Python, los bloques de `if`, `for` y `while` **no crean un scope nuevo**. Una variable asignada en ellos puede seguir disponible al terminar el bloque, siempre que la rama o el ciclo hayan llegado a ejecutarse. Las funciones y los métodos sí crean un alcance local. Las comprensiones (`[x for x in ...]`) manejan su variable iteradora en un alcance propio en Python 3.

8.2 Alcance en funciones: lectura, asignación, `global` y `nonlocal`

Los parámetros y los nombres asignados dentro de una función son **locales por defecto**. Desaparecen al finalizar la llamada, salvo que un objeto sea retornado o quede referenciado desde otro lugar. Una función puede **leer** un nombre global sin ninguna declaración especial; sin embargo, si intenta asignarle un nuevo valor, Python interpretará ese nombre como local. Consultarlo antes de esa asignación puede producir `UnboundLocalError`.

Existen dos declaraciones para cambiar esta regla:

- `global nombre` permite que una función reasigne el nombre ubicado en el alcance global. Debe usarse con moderación, porque introduce estado compartido y hace más difícil seguir los cambios del programa. Con frecuencia es más claro recibir un valor como argumento y devolver el resultado.
- `nonlocal nombre` permite que una función anidada reasigne un nombre del alcance envolvente más cercano. No busca en el alcance global. Es útil en cierres (*closures*) que necesitan conservar un pequeño estado entre llamadas.

Modificar un objeto mutable y reasignar un nombre no son lo mismo. Una función puede ejecutar `lista_global.append(x)` sin declarar `global`, porque el nombre continúa apuntando a la misma lista. Para ejecutar `lista_global = otra_lista`, en cambio, tendría que declarar `global lista_global` si se desea reemplazar el nombre global.

```
# Variable global: puede ser leída por las funciones del notebook
tasa_iva = 0.19


def crear_calculadora_precio(descuento):
    """Crea funciones que calculan precios y conservan un contador.

    Parameters
    ----------
    descuento : float
        Proporción de descuento entre 0 y 1.

    Returns
    -------
    tuple of callable
        Función de cálculo y función de consulta del contador.
    """
    # 'descuento' y 'calculos_realizados' pertenecen al scope envolvente.
    calculos_realizados = 0

    def calcular_precio_final(precio):
        """Calcula el precio con descuento e IVA y actualiza el contador.

        Parameters
        ----------
        precio : float
            Precio antes del descuento y del IVA.

        Returns
        -------
        float
            Precio final después del descuento y del IVA.
        """
        # 'precio', 'subtotal' y 'precio_final' son locales a esta función.
        nonlocal calculos_realizados
        subtotal = precio * (1 - descuento)
        precio_final = subtotal * (1 + tasa_iva)  # Lee la variable global.
        calculos_realizados += 1
        return precio_final

    def mostrar_numero_calculos():
        """Devuelve el número de precios calculados por este cierre.

        Returns
        -------
        int
            Número acumulado de llamadas de cálculo.
        """
        # Lee la variable conservada en el alcance envolvente.
        return calculos_realizados

    return calcular_precio_final, mostrar_numero_calculos


# Cada calculadora conserva su propio descuento y contador.
calcular_estudiante, contar_estudiante = crear_calculadora_precio(0.10)
calcular_general, contar_general = crear_calculadora_precio(0.05)

print("Precio para estudiante:", round(calcular_estudiante(100_000), 2))
print("Otro precio para estudiante:", round(calcular_estudiante(50_000), 2))
print("Cálculos de estudiante:", contar_estudiante())

print("Precio general:", round(calcular_general(100_000), 2))
print("Cálculos generales:", contar_general())
```

```
Precio para estudiante: 107100.0
Otro precio para estudiante: 53550.0
Cálculos de estudiante: 2
Precio general: 113050.0
Cálculos generales: 1
```

8.3 Alcance en clases y objetos

Al ejecutar una sentencia `class`, Python crea un **espacio de nombres de la clase**. Los nombres definidos directamente en su cuerpo se convierten en atributos de clase: por ejemplo, `CuentaAhorro.tasa_interes`. Cada **instancia** posee además su propio espacio de nombres, donde suelen guardarse los atributos asignados mediante `self`, como `cuenta_ana.saldo`. Podemos inspeccionar los atributos de instancia con `vars(objeto)` o `objeto.__dict__` cuando la clase dispone de ese diccionario.

Un método es una función definida en una clase y crea un **alcance local cada vez que se llama**. Sus parámetros, incluido `self`, y sus variables temporales son locales. `self` no es una palabra reservada, sino la convención universal para nombrar la referencia a la instancia que recibió la llamada. Gracias a ella, el método accede explícitamente a `self.saldo` o `self.titular`. En un método de clase, `cls` cumple un papel semejante respecto de la clase.

La búsqueda de `objeto.atributo` no sigue exactamente LEGB: es una **búsqueda de atributos**. De manera simplificada, Python consulta el objeto y después su clase y las clases de las que hereda. Por eso una instancia puede leer `self.tasa_interes` aunque `tasa_interes` esté definida en la clase. Para dejar claro que se desea el valor compartido, suele escribirse `type(self).tasa_interes`, `cls.tasa_interes` o `NombreClase.tasa_interes`.

Hay un matiz importante: el espacio de nombres de la clase **no funciona como alcance envolvente de los métodos**. Dentro de un método no se debe escribir simplemente `tasa_interes`; se debe usar `self.tasa_interes`, `type(self).tasa_interes` o el nombre de la clase.

- **Variable local del método:** `interes`; existe solamente durante esa llamada.
- **Atributo de instancia:** `self.saldo`; cada objeto conserva el suyo.
- **Atributo de clase:** `CuentaAhorro.tasa_interes`; se comparte como valor predeterminado entre las instancias.
- **Variable global:** un nombre definido fuera de la clase y sus métodos; puede ser leído mediante LEGB, aunque es preferible pasar las dependencias de forma explícita cuando forman parte del estado del objeto.

```
# Define una clase sencilla para distinguir scopes de clase e instancia.
class CuentaAhorro:
    """Representa una cuenta de ahorro con una tasa compartida.

    Parameters
    ----------
    titular : str
        Nombre de la persona titular.
    saldo_inicial : float, default=0
        Recursos disponibles al crear la cuenta.

    Attributes
    ----------
    tasa_interes : float
        Tasa compartida por las instancias.
    cuentas_creadas : int
        Número acumulado de instancias.
    titular : str
        Titular guardado en cada instancia.
    saldo : float
        Saldo propio de cada instancia.
    """

    # Atributos de clase: compartidos por todas las instancias.
    tasa_interes = 0.05
    cuentas_creadas = 0

    def __init__(self, titular, saldo_inicial=0):
        """Inicializa una cuenta y actualiza el contador compartido."""
        # Atributos de instancia: cada objeto guarda sus propios valores.
```

```
        self.titular = titular
        self.saldo = saldo_inicial
        type(self).cuentas_creadas += 1

    def aplicar_interes(self):
        """Aplica la tasa vigente al saldo de la cuenta.

        Returns
        -------
        float
            Interés agregado al saldo.
        """
        # Variable local: solo existe durante esta llamada al método.
        interes = self.saldo * type(self).tasa_interes
        self.saldo += interes
        return interes

    @classmethod
    def cambiar_tasa(cls, nueva_tasa):
        """Cambia la tasa de interés compartida por la clase.

        Parameters
        ----------
        nueva_tasa : float
            Nueva tasa expresada como proporción.
        """
        # Reasigna el atributo de clase mediante cls.
        cls.tasa_interes = nueva_tasa


# Crea dos objetos con atributos de instancia independientes.
cuenta_ana = CuentaAhorro("Ana", 1_000_000)
cuenta_luis = CuentaAhorro("Luis", 500_000)

print("Atributos de Ana:", vars(cuenta_ana))
print("Atributos de Luis:", vars(cuenta_luis))
print("Cuentas creadas:", CuentaAhorro.cuentas_creadas)

interes_ana = cuenta_ana.aplicar_interes()
print("Interés de Ana:", interes_ana)
print("Nuevo saldo de Ana:", cuenta_ana.saldo)
print("El saldo de Luis no cambió:", cuenta_luis.saldo)

# El método de clase cambia la tasa compartida.
CuentaAhorro.cambiar_tasa(0.06)
print("Nueva tasa desde Ana:", cuenta_ana.tasa_interes)
print("Nueva tasa desde Luis:", cuenta_luis.tasa_interes)
```

```
Atributos de Ana: {'titular': 'Ana', 'saldo': 1000000}
Atributos de Luis: {'titular': 'Luis', 'saldo': 500000}
Cuentas creadas: 2
Interés de Ana: 50000.0
Nuevo saldo de Ana: 1050000.0
El saldo de Luis no cambió: 500000
Nueva tasa desde Ana: 0.06
Nueva tasa desde Luis: 0.06
```

8.4 Alcance global en Jupyter y buenas prácticas

En Jupyter, las variables, funciones, clases e importaciones creadas en una celda permanecen en el **espacio global del kernel** y pueden utilizarse en otras celdas. El orden visual no garantiza el orden de ejecución: si se ejecutan celdas fuera de orden, un nombre puede conservar un valor antiguo o todavía no existir. Ante resultados extraños, conviene reiniciar el kernel y ejecutar todas las celdas desde el comienzo.

Como reglas prácticas: use nombres locales para resultados temporales; pase la información mediante parámetros y `return`; almacene en `self` aquello que pertenece al estado de un objeto; reserve los atributos de clase para información realmente compartida; y utilice `global` o `nonlocal` solamente cuando el estado compartido sea

intencional. Estas decisiones reducen efectos secundarios y hacen que el origen de cada valor sea más fácil de identificar.

---

9. Temas adicionales de Python

↑ Volver al índice

En esta sección estudiaremos tres herramientas que permiten escribir programas más flexibles y expresivos: argumentos variables en funciones, enumeraciones y recorridos paralelos de colecciones.

9.1 Argumentos variables con `*args` y `**kwargs`

Una función con una lista fija de parámetros conoce de antemano cuántos valores recibirá. Cuando una función debe aceptar una cantidad variable, Python permite **empaquetar argumentos adicionales** mediante uno o dos asteriscos:

- `*args` reúne los argumentos **posicionales** adicionales dentro de una **tupla**. El nombre `args` es una convención; el comportamiento proviene del asterisco. Puede recibir cero, uno o muchos valores.
- `**kwargs` reúne los argumentos adicionales escritos como `clave=valor` dentro de un **diccionario**. `kwargs` significa *keyword arguments* y también es un nombre convencional; los dos asteriscos producen el empaquetamiento.

En una definición completa, el orden habitual es `def funcion(parametros_normales, *args, parametros_solo_por_nombre, **kwargs):`. Todo parámetro declarado después de `*args` debe proporcionarse por nombre. `**kwargs` aparece al final porque captura los argumentos por palabra clave que no fueron asignados a parámetros anteriores. La documentación oficial sobre funciones desarrolla estas formas de llamada.

Los asteriscos realizan la operación inversa durante una llamada: `funcion(*secuencia)` **desempaqueta** una lista o tupla como argumentos posicionales y `funcion(**diccionario)` desempaqueta sus pares como argumentos por nombre. Las claves de ese diccionario deben ser cadenas válidas para los parámetros y no pueden proporcionar dos veces el mismo argumento.

> `*args` y `**kwargs` son útiles en funciones envolventes, constructores configurables y APIs extensibles, pero no deben reemplazar parámetros explícitos cuando estos son conocidos: una firma explícita es más fácil de leer, validar y documentar.

```
# Define una función flexible con argumentos posicionales y nominales variables.
def resumir_notas(estudiante, *notas, decimales=2, **metadatos):
    """Resume las notas y los metadatos de un estudiante.

    Parameters
    ----------
    estudiante : str
        Nombre del estudiante.
    *notas : float
        Cantidad variable de calificaciones posicionales.
    decimales : int, default=2
        Número de decimales utilizado para redondear el promedio.
    **metadatos : object
        Información adicional proporcionada por palabra clave.

    Returns
    -------
    dict
```

```
        Nombre, notas, promedio y metadatos del estudiante.

    Raises
    ------
    ValueError
        Si no se proporciona ninguna nota.
    """
    # args se recibe como una tupla; aquí exigimos al menos una nota.
    if not notas:
        raise ValueError("Debes proporcionar al menos una nota.")

    # Calcula el promedio y conserva kwargs como un diccionario.
    promedio = round(sum(notas) / len(notas), decimales)
    return {
        "estudiante": estudiante,
        "notas": notas,
        "promedio": promedio,
        "metadatos": metadatos,
    }


# Prepara colecciones que se desempaquetarán al llamar la función.
notas_ana = (4.2, 4.7, 3.9)
informacion_ana = {"programa": "Economía", "semestre": 5}

# * desempaqueta la tupla y ** desempaqueta el diccionario.
resumen_ana = resumir_notas(
    "Ana",
    *notas_ana,
    decimales=1,
    **informacion_ana,
)

print("Notas recibidas por *args:", resumen_ana["notas"])
print("Promedio:", resumen_ana["promedio"])
print("Datos recibidos por **kwargs:", resumen_ana["metadatos"])
```

```
Notas recibidas por *args: (4.2, 4.7, 3.9)
Promedio: 4.3
Datos recibidos por **kwargs: {'programa': 'Economía', 'semestre': 5}
```

9.2 Enumeraciones con `Enum`

Una **enumeración** es un conjunto finito de nombres simbólicos asociados con valores constantes. El módulo `enum` pertenece a la biblioteca estándar. Para crear una enumeración se define una clase que hereda de `Enum`; por convención, los nombres de sus miembros se escriben en mayúsculas.

Las enumeraciones son preferibles a cadenas o números dispersos cuando una variable solo puede adoptar ciertos estados. `EstadoModelo.ESTIMADO` comunica mejor la intención que la cadena `"estimado"`, evita errores ortográficos y permite que el editor sugiera las opciones válidas. Cada miembro dispone de `.name`, que contiene su nombre, y `.value`, que contiene el valor asociado.

Los valores pueden asignarse manualmente o generarse con `auto()` cuando solo importa que cada miembro sea distinto. Una enumeración es iterable: un ciclo `for` recorre sus miembros canónicos en el orden de definición. Los miembros se comparan entre sí y, como representan objetos únicos, es habitual usar `is` para verificar un estado específico.

```
# Importa las herramientas de enumeración desde la biblioteca estándar.
from enum import Enum, auto


# Define los únicos estados permitidos para un modelo econométrico.
class EstadoModelo(Enum):
    """Representa las etapas posibles de un modelo econométrico."""

    SIN_ESTIMAR = auto()
```

```
    ESTIMADO = auto()
    VALIDADO = auto()


# Un ciclo recorre todos los miembros en el orden de definición.
for estado in EstadoModelo:
    print("Nombre:", estado.name, "- Valor:", estado.value)

# Asigna y compara un miembro concreto de la enumeración.
estado_actual = EstadoModelo.ESTIMADO

if estado_actual is EstadoModelo.ESTIMADO:
    print("El modelo ya fue estimado y puede validarse.")
```

```
Nombre: SIN_ESTIMAR - Valor: 1
Nombre: ESTIMADO - Valor: 2
Nombre: VALIDADO - Valor: 3
El modelo ya fue estimado y puede validarse.
```

9.3 Recorridos paralelos con `zip()`

La función incorporada `zip()` recibe dos o más iterables y los recorre **en paralelo**. En cada posición produce una tupla con un elemento de cada iterable. Es especialmente útil dentro de un ciclo `for`, donde esa tupla puede desempaquetarse directamente en varias variables.

`zip()` devuelve un **iterador perezoso**: los pares no se calculan todos de inmediato, sino a medida que el ciclo los solicita. Por eso, después de consumirlo una vez, el mismo iterador queda agotado; si se necesita conservar su contenido, puede convertirse en `list`.

Por defecto, `zip()` se detiene cuando termina el iterable más corto e ignora los elementos sobrantes de los demás. Cuando se espera que todas las colecciones tengan la misma longitud, conviene usar `strict=True`: Python producirá `ValueError` si las longitudes difieren, en lugar de ocultar potencialmente un error en los datos.

También es posible construir un diccionario a partir de pares con `dict(zip(claves, valores))` o realizar la operación inversa mediante `zip(*pares)`, que separa nuevamente las columnas.

```
# Define tres colecciones relacionadas por su posición.
variables = ["Intercepto", "GNP", "UNEMP"]
coeficientes = [50_000.0, 0.04, -0.02]
errores_estandar = [8_000.0, 0.01, 0.008]

# zip() permite recorrer y desempaquetar las tres colecciones en paralelo.
for variable, coeficiente, error in zip(
    variables, coeficientes, errores_estandar, strict=True
):
    print(
        f"{variable}: coeficiente={coeficiente}, error estándar={error}"
    )

# Materializa pares y construye un diccionario variable-coeficiente.
pares_coeficientes = list(zip(variables, coeficientes, strict=True))
coeficiente_por_variable = dict(pares_coeficientes)
print("Diccionario de coeficientes:", coeficiente_por_variable)

# El asterisco desempaqueta los pares para separar nuevamente las columnas.
variables_recuperadas, coeficientes_recuperados = zip(*pares_coeficientes)
print("Variables recuperadas:", variables_recuperadas)
print("Coeficientes recuperados:", coeficientes_recuperados)
```

```
Intercepto: coeficiente=50000.0, error estándar=8000.0
GNP: coeficiente=0.04, error estándar=0.01
UNEMP: coeficiente=-0.02, error estándar=0.008
Diccionario de coeficientes: {'Intercepto': 50000.0, 'GNP': 0.04, 'UNEMP': -0.02}
Variables recuperadas: ('Intercepto', 'GNP', 'UNEMP')
Coeficientes recuperados: (50000.0, 0.04, -0.02)
```

10. CPython y la biblioteca estándar de Python

↑ Volver al índice

Python es un lenguaje que puede tener distintas implementaciones. **CPython** es la implementación de referencia y el intérprete más utilizado; está escrito principalmente en C. Su distribución incluye tanto el intérprete como la **biblioteca estándar de Python** (*Python Standard Library*): un conjunto amplio de módulos y paquetes disponibles sin instalar dependencias externas. Algunos están escritos en Python y otros, por razones de rendimiento o acceso al sistema, en C. Esta colección suele resumirse con la idea de que Python viene con *batteries included*. Puedes consultar la documentación oficial de la biblioteca estándar.

Entre sus módulos más útiles se encuentran:

- **Archivos, rutas y sistema:** `pathlib` ofrece una forma moderna de manejar rutas; `os` permite interactuar con el sistema operativo; y `sys` da acceso al intérprete, sus argumentos y opciones.
- **Matemáticas y azar:** `math` aporta funciones y constantes matemáticas; `random` genera números pseudoaleatorios; y `statistics` calcula estadísticas descriptivas básicas.
- **Fechas y horas:** `datetime` permite representar y operar con fechas, horas y duraciones.
- **Colecciones e iteración:** `collections` incorpora estructuras especializadas; `itertools` construye iteradores eficientes; y `functools` ofrece utilidades para trabajar con funciones.
- **Datos y texto:** `json` y `csv` permiten leer y escribir formatos comunes, mientras que `re` sirve para buscar patrones mediante expresiones regulares.
- **Programas y diagnóstico:** `subprocess` ejecuta otros programas y `logging` registra eventos de una aplicación de forma configurable.
- **Anotaciones de tipos:** `typing` contiene herramientas para documentar los tipos esperados por el código y facilitar su análisis.

La biblioteca estándar forma parte de la instalación de Python. En cambio, librerías como `NumPy`, `pandas`, `matplotlib`, `requests`, `SciPy` y `scikit-learn` son paquetes de terceros y normalmente se instalan por separado, por ejemplo con `pip` o `conda`.

11. Librerías para análisis de datos y econometría

↑ Volver al índice

A diferencia de la biblioteca estándar, las siguientes son **librerías de terceros** que normalmente se instalan por separado. Juntas forman una parte central del ecosistema de análisis de datos en Python:

- **NumPy** (`numpy`, alias habitual `np`): ofrece arreglos multidimensionales (`ndarray`) y operaciones numéricas eficientes, como álgebra lineal y cálculos estadísticos básicos.
- **SciPy** (`scipy`): amplía NumPy con algoritmos científicos especializados para optimización, integración, álgebra lineal, estadística, procesamiento de señales y matrices dispersas.
- **pandas** (`pandas`, alias `pd`): permite organizar, limpiar, transformar y analizar datos tabulares mediante estructuras como `Series` y `DataFrame`.
- **Matplotlib** (`matplotlib.pyplot`, alias `plt`): es una biblioteca flexible para crear gráficos y personalizar sus elementos con detalle.
- **Seaborn** (`seaborn`, alias `sns`): construye sobre Matplotlib y ofrece una interfaz de alto nivel para producir gráficos estadísticos informativos y atractivos.

- **statsmodels** (`statsmodels.api`, alias `sm`): contiene herramientas para estimar modelos estadísticos y econométricos, realizar pruebas de hipótesis y obtener diagnósticos y tablas de resultados.

En un flujo de trabajo típico, NumPy realiza los cálculos numéricos, SciPy aporta algoritmos científicos especializados, pandas organiza los datos, Matplotlib y Seaborn los visualizan, y statsmodels permite estimar y evaluar los modelos.

---

12. Importar una librería: NumPy

↑ Volver al índice

NumPy (*Numerical Python*) es la librería fundamental para computación numérica en Python. Su objeto central es `ndarray`, un arreglo homogéneo de una o más dimensiones que permite almacenar y manipular grandes cantidades de datos numéricos de forma eficiente. Por convención, NumPy se importa con el alias `np`.

NumPy permite seleccionar, reorganizar y transformar arreglos, además de aplicar operaciones **vectorizadas**: una sola expresión opera sobre muchos elementos sin escribir un ciclo de Python para cada uno. También incluye `numpy.linalg`, un submódulo con herramientas de álgebra lineal para productos matriciales, sistemas de ecuaciones, determinantes, normas, descomposiciones y valores propios.

El álgebra lineal es central en econometría porque los datos y los modelos suelen representarse mediante vectores y matrices. Por ejemplo, la expresión matricial del estimador MCO es

$$\widehat{\boldsymbol{\beta}} = (X^{\mathsf{T}}X)^{-1}X^{\mathsf{T}}\mathbf{y}.$$

En aplicaciones numéricas suele ser preferible resolver el sistema correspondiente con `solve()` en lugar de calcular una matriz inversa de manera explícita, pues generalmente es más estable y eficiente.

```
# Importa NumPy con el alias convencional np para usarlo en esta sección.
import numpy as np
```

12.1 El objeto `ndarray`: la estructura central de NumPy

Un `ndarray` (*N-dimensional array*) es un arreglo rectangular de N dimensiones cuyos elementos normalmente comparten el mismo tipo de dato. Un arreglo de cero dimensiones contiene un escalar; uno de una dimensión puede representar un vector; uno de dos dimensiones, una matriz; y uno de tres o más dimensiones permite organizar colecciones de matrices u otros datos multidimensionales. Cada dimensión también recibe el nombre de **eje** (*axis*).

A diferencia de una lista de Python, que puede mezclar objetos de tipos distintos y guardar referencias separadas, un `ndarray` organiza datos homogéneos en un bloque de memoria con una estructura bien definida. Su tamaño total permanece fijo, aunque su forma puede reinterpretarse cuando el número de elementos no cambia. Esta organización permite que muchas operaciones se ejecuten en código compilado, con menor consumo de memoria y sin recorrer cada elemento mediante ciclos escritos manualmente en Python.

¿Por qué son tan importantes?

- **Vectorización:** expresiones como `arreglo * 2` o `arreglo_a + arreglo_b` operan sobre muchos elementos a la vez y producen código más conciso.
- **Eficiencia:** almacenar un único `dtype` permite aprovechar mejor la memoria y rutinas numéricas optimizadas.

- **Varias dimensiones:** los ejes permiten representar observaciones, variables, periodos, simulaciones o matrices con una sola estructura.
- **Broadcasting:** NumPy puede combinar arreglos de formas diferentes, pero compatibles, extendiendo conceptualmente la dimensión más pequeña sin copiar todos sus valores.
- **Interoperabilidad:** los arreglos de NumPy sirven de base para buena parte del ecosistema científico de Python, incluidas SciPy, pandas, statsmodels y scikit-learn.

Argumentos importantes de `np.array()`

La forma habitual de construir un arreglo es `np.array(object, dtype=None, copy=True, order='K', ndmin=0)`. Sus argumentos más útiles al comenzar son:

- `object`: los datos originales, por ejemplo una lista, una lista de listas u otro arreglo.
- `dtype`: el tipo de todos los elementos, como `int`, `float`, `np.float64` o `bool`. Si se omite, NumPy infiere un tipo compatible con los datos.
- `copy`: con `True` se fuerza la creación de una copia independiente. Esto evita que ciertas modificaciones compartan accidentalmente los mismos datos.
- `ndmin`: establece el número mínimo de dimensiones del resultado; por ejemplo, `ndmin=2` garantiza al menos dos ejes.
- `order`: controla la disposición de los datos en memoria. El valor habitual `'K'` intenta conservar el orden original; `'C'` prioriza el orden por filas y `'F'` el orden por columnas. En ejemplos introductorios rara vez es necesario modificarlo.

NumPy también ofrece funciones de creación como `np.zeros()`, `np.ones()`, `np.arange()`, `np.linspace()` y `np.eye()` para construir arreglos con ceros, unos, secuencias, puntos equidistantes o matrices identidad.

Atributos esenciales

Los atributos describen el arreglo y se consultan **sin paréntesis**:

- `.ndim`: número de dimensiones o ejes.
- `.shape`: tupla con la longitud de cada eje; `(2, 3)` significa dos filas y tres columnas.
- `.size`: número total de elementos.
- `.dtype`: tipo de dato almacenado.
- `.itemsize`: bytes utilizados por cada elemento; `.nbytes`: bytes ocupados por todos los elementos.
- `.T`: vista transpuesta del arreglo.

Métodos frecuentes

Los métodos realizan operaciones y se llaman **con paréntesis**:

- `.reshape(nueva_forma)`: reorganiza los mismos elementos en una forma compatible. El valor `-1` permite que NumPy infiera una dimensión.
- `.astype(tipo)`: devuelve los datos convertidos a otro `dtype`.
- `.copy()`: crea una copia independiente que puede modificarse sin alterar el arreglo original.
- `.sum()`, `.mean()`, `.min()`, `.max()`, `.std()` y `.var()`: calculan resúmenes numéricos. El argumento `axis=0` opera a lo largo de las filas y entrega un resultado por columna; `axis=1` opera a lo largo de las columnas y entrega un resultado por fila.
- `.ravel()`: presenta los elementos en una dimensión cuando es posible; `.flatten()` produce siempre una copia unidimensional.
- `.sort()`: ordena el arreglo en el lugar, mientras que `np.sort(arreglo)` devuelve una versión ordenada sin modificar necesariamente el original.

Además de los métodos, la sintaxis `arreglo[fila, columna]` selecciona un elemento y `arreglo[:, columna]` selecciona una columna completa. Los índices empiezan en cero, como en las listas de Python.

```
# Define datos tabulares mediante una lista de listas.
datos_base = [[1, 2, 3],
              [4, 5, 6]]

# Crea un ndarray bidimensional de float y copia los datos de origen.
arreglo = np.array(
    datos_base, dtype=np.float64, copy=True, ndmin=2
)

# Inspecciona los principales atributos que describen el arreglo.
print("Tipo de objeto:", type(arreglo))
print("Dimensiones:", arreglo.ndim)
print("Forma:", arreglo.shape)
print("Número de elementos:", arreglo.size)
print("Tipo de dato:", arreglo.dtype)
print("Bytes por elemento:", arreglo.itemsize)
print("Bytes totales:", arreglo.nbytes)

# Selecciona un elemento y una columna mediante indexación.
print("Elemento de la primera fila y segunda columna:", arreglo[0, 1])
print("Segunda columna:", arreglo[:, 1])

# Reorganiza, aplana y convierte el arreglo sin alterar el original.
print("Nueva forma (3, 2):")
print(arreglo.reshape(3, 2))
print("Arreglo en una dimensión:", arreglo.ravel())
print("Conversión a enteros:")
print(arreglo.astype(np.int64))

# Resume los datos completos o a lo largo de un eje específico.
print("Suma total:", arreglo.sum())
print("Suma por columnas (axis=0):", arreglo.sum(axis=0))
print("Promedio por filas (axis=1):", arreglo.mean(axis=1))

# Resta automáticamente cada media a su columna mediante broadcasting.
promedios_columnas = arreglo.mean(axis=0)
arreglo_centrado = arreglo - promedios_columnas
print("Columnas centradas mediante broadcasting:")
print(arreglo_centrado)

# Crea una copia independiente y demuestra que el original no cambia.
copia = arreglo.copy()
copia[0, 0] = 99
print("El arreglo original no cambia al modificar la copia:")
print(arreglo)
```

```
Tipo de objeto: <class 'numpy.ndarray'>
Dimensiones: 2
Forma: (2, 3)
Número de elementos: 6
Tipo de dato: float64
Bytes por elemento: 8
Bytes totales: 48
Elemento de la primera fila y segunda columna: 2.0
Segunda columna: [2. 5.]
Nueva forma (3, 2):
[[1. 2.]
 [3. 4.]
 [5. 6.]]
Arreglo en una dimensión: [1. 2. 3. 4. 5. 6.]
Conversión a enteros:
[[1 2 3]
 [4 5 6]]
Suma total: 21.0
Suma por columnas (axis=0): [5. 7. 9.]
Promedio por filas (axis=1): [2. 5.]
Columnas centradas mediante broadcasting:
[[-1.5 -1.5 -1.5]
 [ 1.5  1.5  1.5]]
```

```
El arreglo original no cambia al modificar la copia:
[[1. 2. 3.]
 [4. 5. 6.]]
```

12.2 Creación de vectores y matrices

Un arreglo de una dimensión puede representar un vector y uno de dos dimensiones puede representar una matriz. Además de `np.array()`, usaremos `np.zeros()` para crear una matriz de ceros y `np.eye()` para construir una matriz identidad.

```
# Crea un vector unidimensional de números de punto flotante.
vector = np.array([1, 2, 3], dtype=float)

# Crea dos matrices de 2 × 2 a partir de listas anidadas.
matriz_a = np.array([[1, 2],
                     [3, 4]], dtype=float)

matriz_b = np.array([[5, 6],
                     [7, 8]], dtype=float)

# Genera arreglos especiales: una matriz de ceros y una identidad.
matriz_ceros = np.zeros((2, 3))
matriz_identidad = np.eye(3)

# Muestra las formas de datos creadas para compararlas.
print("Vector:", vector)
print("Matriz A:")
print(matriz_a)
print("Matriz B:")
print(matriz_b)
print("Matriz de ceros:")
print(matriz_ceros)
print("Matriz identidad:")
print(matriz_identidad)
```

```
Vector: [1. 2. 3.]
Matriz A:
[[1. 2.]
 [3. 4.]]
Matriz B:
[[5. 6.]
 [7. 8.]]
Matriz de ceros:
[[0. 0. 0.]
 [0. 0. 0.]]
Matriz identidad:
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

12.3 Manipulación y operaciones elemento a elemento

NumPy permite seleccionar filas o columnas mediante índices, cambiar la forma con `reshape()` y aplicar operaciones a todos los elementos. Los operadores `+`, `-`, `*` y `/` actúan elemento a elemento cuando los arreglos tienen formas compatibles.

```
# Suma y multiplica las matrices elemento a elemento.
suma_matrices = matriz_a + matriz_b
producto_elemento_a_elemento = matriz_a * matriz_b
# Selecciona la primera columna: ':' conserva todas las filas.
primera_columna = matriz_a[:, 0]
# Genera seis números y los reorganiza como una matriz de 2 × 3.
matriz_reorganizada = np.arange(1, 7).reshape(2, 3)

# Presenta los resultados de cada transformación.
print("Suma:")
print(suma_matrices)
```

```
print("Producto elemento a elemento:")
print(producto_elemento_a_elemento)
print("Primera columna:", primera_columna)
print("Arreglo reorganizado:")
print(matriz_reorganizada)
```

```
Suma:
[[ 6.  8.]
 [10. 12.]]
Producto elemento a elemento:
[[ 5. 12.]
 [21. 32.]]
Primera columna: [1. 3.]
Arreglo reorganizado:
[[1 2 3]
 [4 5 6]]
```

12.4 Operaciones matriciales

El operador `@` realiza la multiplicación matricial, mientras que `*` multiplica elemento a elemento. El atributo `.T` transpone una matriz, es decir, intercambia sus filas y columnas. Para que AB exista, el número de columnas de A debe coincidir con el número de filas de B.

```
# '@' calcula el producto matricial entre A y B.
producto_matrices = matriz_a @ matriz_b
# '.T' intercambia las filas y las columnas de A.
matriz_transpuesta = matriz_a.T

# Muestra ambos resultados para diferenciarlos de operaciones elemento a elemento.
print("Producto matricial:")
print(producto_matrices)
print("Transpuesta de A:")
print(matriz_transpuesta)
```

```
Producto matricial:
[[19. 22.]
 [43. 50.]]
Transpuesta de A:
[[1. 3.]
 [2. 4.]]
```

12.5 Resolver sistemas de ecuaciones lineales

Un sistema de n ecuaciones lineales con n incógnitas puede escribirse como

$$A\mathbf{x} = \mathbf{b},$$

donde A es la matriz de coeficientes, $\mathbf{x}$ es el vector de incógnitas y $\mathbf{b}$ contiene los términos independientes. `np.linalg.solve(A, b)` encuentra $\mathbf{x}$ cuando A es cuadrada y no singular. Después verificamos la solución comprobando que $A\mathbf{x}$ sea igual a $\mathbf{b}$ dentro de la tolerancia numérica.

```
# Construye la matriz de coeficientes A del sistema lineal.
A_sistema = np.array([[3, 1],
                      [1, 2]], dtype=float)
# Define el vector de términos independientes b.
b_sistema = np.array([9, 8], dtype=float)

# Resuelve A @ x = b sin calcular explícitamente la inversa de A.
solucion = np.linalg.solve(A_sistema, b_sistema)

# Sustituye la solución y verifica numéricamente el resultado.
print("Solución x:", solucion)
print("A @ x:", A_sistema @ solucion)
print("La solución satisface el sistema:",
      np.allclose(A_sistema @ solucion, b_sistema))
```

```
Solución x: [2. 3.]
A @ x: [9. 8.]
La solución satisface el sistema: True
```

12.6 Valores y vectores propios

Un vector no nulo $\mathbf{v}$ es un **vector propio** de A si al multiplicarlo por la matriz solamente cambia su escala:

$$A\mathbf{v} = \lambda\mathbf{v},$$

donde λ es el **valor propio** asociado. `np.linalg.eig(A)` devuelve los valores propios y una matriz cuyas columnas son los vectores propios correspondientes. Estas cantidades aparecen, por ejemplo, en el análisis de estabilidad, componentes principales y descomposiciones matriciales. Para matrices reales simétricas también existe `np.linalg.eigh()`, una rutina especializada.

```
# Define la matriz cuadrada cuyos valores propios se calcularán.
A_propios = np.array([[2, 1],
                      [1, 2]], dtype=float)

# eig() devuelve los valores propios y los vectores propios por columnas.
valores_propios, vectores_propios = np.linalg.eig(A_propios)

print("Valores propios:", valores_propios)
print("Vectores propios (por columnas):")
print(vectores_propios)

# Extrae el primer par propio y comprueba A @ v = lambda * v.
primer_valor = valores_propios[0]
primer_vector = vectores_propios[:, 0]
print("Se cumple A @ v = lambda * v:",
      np.allclose(A_propios @ primer_vector,
                  primer_valor * primer_vector))
```

```
Valores propios: [3. 1.]
Vectores propios (por columnas):
[[ 0.70710678 -0.70710678]
 [ 0.70710678  0.70710678]]
Se cumple A @ v = lambda * v: True
```

12.7 Complemento con SciPy: matrices dispersas

Los ejemplos anteriores usan matrices pequeñas y densas, por lo que NumPy es suficiente. Usamos **SciPy** en el siguiente ejemplo porque incorpora estructuras y algoritmos especializados para **matrices dispersas**, es decir, matrices en las que la mayoría de los elementos son cero. Almacenar solamente los valores distintos de cero puede ahorrar mucha memoria y acelerar los cálculos en problemas grandes.

La clase `csr_matrix` crea una representación dispersa y `spsolve()` resuelve el sistema sin convertir la matriz a una representación densa. Para el sistema pequeño de la sección 12.5, `np.linalg.solve()` sigue siendo la opción más sencilla.

```
# Importa la estructura CSR y el solucionador para matrices dispersas.
from scipy.sparse import csr_matrix
from scipy.sparse.linalg import spsolve

# Almacena en formato disperso la matriz de coeficientes.
A_dispersa = csr_matrix([[4, 1, 0],
                         [1, 4, 1],
                         [0, 1, 3]], dtype=float)
# Define el vector independiente del sistema disperso.
b_dispersa = np.array([15, 15, 10], dtype=float)

# Resuelve el sistema sin convertir la matriz a formato denso.
solucion_dispersa = spsolve(A_dispersa, b_dispersa)
```

```
# Verifica que la solución reconstruya aproximadamente el vector b.
print("Solución del sistema disperso:", solucion_dispersa)
print("La solución satisface el sistema:",
      np.allclose(A_dispersa @ solucion_dispersa, b_dispersa))
```

```
Solución del sistema disperso: [3.17073171 2.31707317 2.56097561]
La solución satisface el sistema: True
```

Resumen

↑ Volver al índice

En este cuaderno aprendimos a:

- Explicar qué es Python, sus principales usos y las herramientas locales utilizadas en el curso.
- Diferenciar scripts `.py`, notebooks `.ipynb` y documentos reproducibles `.qmd`.
- Distinguir paradigmas de programación, formas de ejecución y sistemas de tipado.
- Aplicar comentarios y las principales convenciones de estilo de PEP 8.
- Crear variables con diferentes tipos de datos.
- Identificar tipos mediante `type()`.
- Realizar operaciones aritméticas y lógicas.
- Distinguir objetos mutables e inmutables, indexar colecciones y usar los principales métodos de listas, tuplas y diccionarios.
- Usar condicionales y ciclos, distinguiendo variables contadoras y acumuladoras.
- Documentar código mediante comentarios y docstrings compatibles con PEP 257 y el estilo de NumPy/SciPy.
- Implementar un modelo MCO con funciones imperativas y con una clase orientada a objetos, reconociendo las diferencias entre ambos enfoques.
- Comprender la regla LEGB y distinguir el alcance de variables locales, envolventes, globales, de clase y de instancia.
- Utilizar `*args`, `**kwargs`, enumeraciones y `zip()` para construir funciones y recorridos más flexibles.
- Reconocer la relación entre CPython y la biblioteca estándar, junto con algunos de sus módulos principales.
- Identificar el papel de NumPy, SciPy, pandas, Matplotlib, Seaborn y statsmodels en el análisis de datos.
- Manipular arreglos con NumPy y realizar operaciones de álgebra lineal, como resolver sistemas y calcular valores y vectores propios.
- Reconocer cuándo SciPy resulta apropiado para trabajar con matrices dispersas.